

Contents

第一章 **vector**（重点）

1. **vector** 定义

名称	1
用法	1
作用	1

2. **vector** 初始化

名称	2
用法	2
作用	2

3. **push_back()**

名称	2
用法	2
作用	2

4. **pop_back()**

名称	2
用法	2
作用	2

5. **size()**

名称	2
用法	2
作用	2

6. **capacity()**

名称	3
用法	3
作用	3

7. **empty()**

名称	3
用法	3
作用	3

8. **clear()**

名称	3
用法	3
作用	3

9. **front()**

名称	3
用法	3
作用	3

10. **back()**

名称	3
----------	---

用法	3
----------	---

作用	4
----------	---

11. 下标访问 []

名称	4
----------	---

用法	4
----------	---

作用	4
----------	---

12. **at()**

名称	4
----------	---

用法	4
----------	---

作用	4
----------	---

13. **begin()**

名称	4
----------	---

用法	4
----------	---

作用	4
----------	---

14. **end()**

名称	4
----------	---

用法	4
----------	---

作用	4
----------	---

15. **rbegin()**

名称	4
----------	---

用法	4
----------	---

作用	4
----------	---

16. **rend()**

名称	4
----------	---

用法	5
----------	---

作用	5
----------	---

17. **insert()**

名称	5
----------	---

用法	5
----------	---

作用	5
----------	---

18. **erase()**

名称	5
----------	---

用法	5
----------	---

作用	5
----------	---

19. **resize()**

名称	5
----------	---

用法	5
----------	---

作用	5
----------	---

20. **reserve()**

名称	5
----------	---

用法	5
----------	---

作用	5	名称	7
21. swap()	6	用法	7
名称	6	作用	8
用法	6	7. at()	8
作用	6	名称	8
22. assign()	6	用法	8
名称	6	作用	8
用法	6	8. front()	8
作用	6	名称	8
23. data()	6	用法	8
名称	6	作用	8
用法	6	9. back()	8
作用	6	名称	8
vector 遍历（考试必背）	6	用法	8
方法 1: 下标	6	作用	8
方法 2: 范围 for	6	10. push_back()	8
方法 3: 迭代器	6	名称	8
方法 4: 反向	6	用法	8
第二章 string（重点）	6	作用	8
1. string 定义	6	11. pop_back()	8
名称	6	名称	8
用法	6	用法	9
作用	7	作用	9
2. string 输入	7	12. append()	9
方法 1: cin（输入一个单词）	7	名称	9
方法 2: getline（输入整行）	7	用法	9
作用	7	作用	9
3. size()	7	13. 字符串连接 +	9
名称	7	名称	9
用法	7	用法	9
作用	7	作用	9
4. length()	7	14. insert()	9
名称	7	名称	9
用法	7	用法	9
作用	7	作用	9
5. empty()	7	15. erase()	9
名称	7	名称	9
用法	7	用法 1: 删除指定位置	9
作用	7	用法 2: 删除指定长度	10
6. 字符访问 []	7	作用	10
		16. clear()	10

名称	10	用法	12
用法	10	作用	12
作用	10		
17. substr()	10	27. stol()	12
名称	10	名称	12
用法	10	用法	12
作用	10	作用	12
18. find()	10	28. stod()	12
名称	10	名称	12
用法	10	用法	12
作用	10	作用	12
19. rfind()	10	29. to_string()	12
名称	10	名称	12
用法	10	用法	12
作用	11	作用	12
20. replace()	11	30. sort 字符串	12
名称	11	名称	12
用法	11	用法	12
作用	11	作用	12
21. compare()	11	31. reverse 字符串	12
名称	11	名称	12
用法	11	用法	13
作用	11	作用	13
22. begin()	11	string 考试高频组合	13
名称	11	1. 判断回文字符串	13
用法	11	2. 统计字符数量	13
作用	11	3. 删除空格	13
23. end()	11	第三章 algorithm (STL 算法库, 考试核心)	13
名称	11	1. sort()	13
用法	11	名称	13
作用	11	用法	13
24. rbegin()	11	作用	13
名称	11	2. stable_sort()	13
用法	11	名称	13
作用	11	用法	14
25. rend()	12	作用	14
名称	12	3. reverse()	14
用法	12	名称	14
作用	12	用法	14
26. stoi()	12	作用	14
名称	12	4. rotate()	14

名称	14	用法	17
用法	14	作用	17
作用	14		
5. swap()	14	15. unique()	17
名称	14	名称	17
用法	14	用法	17
作用	14	作用	17
6. max()	14	16. lower_bound()	17
名称	14	名称	17
用法	14	用法	17
作用	15	作用	17
7. min()	15	17. upper_bound()	17
名称	15	名称	17
用法	15	用法	17
作用	15	作用	17
8. max_element()	15	18. binary_search()	18
名称	15	名称	18
用法	15	用法	18
作用	15	作用	18
9. min_element()	15	19. next_permutation()	18
名称	15	名称	18
用法	15	用法	18
作用	15	作用	18
10. find()	15	20. prev_permutation()	18
名称	15	名称	18
用法	15	用法	18
作用	16	作用	18
11. count()	16	algorithm 高频考试组合	18
名称	16	1. vector 排序去重	18
用法	16	2. 求最大最小值	18
作用	16	3. 求和	18
		4. 判断存在	18
12. accumulate()	16	第四章 deque / stack / queue / priority_queue	18
名称	16	一、deque（双端队列）	18
用法	16	1. deque 定义	18
作用	16	名称	18
13. fill()	16	用法	18
名称	16	作用	19
用法	16	2. push_back()	19
作用	16	名称	19
14. replace()	16	用法	19
名称	16	作用	19

3. push_front()	19	3. pop()	20
名称	19	名称	20
用法	19	用法	20
作用	19	作用	20
4. pop_back()	19	4. top()	21
名称	19	名称	21
用法	19	用法	21
作用	19	作用	21
5. pop_front()	19	5. size()	21
名称	19	名称	21
用法	19	用法	21
作用	19	作用	21
6. front()	19	6. empty()	21
名称	19	名称	21
用法	19	用法	21
作用	19	作用	21
7. back()	19	stack 经典模板	21
名称	19	三、queue（普通队列）	21
用法	19	1. queue 定义	21
作用	20	名称	21
8. size()	20	用法	21
名称	20	作用	21
用法	20	2. push()	21
作用	20	名称	21
9. empty()	20	用法	21
名称	20	作用	21
用法	20	3. pop()	21
作用	20	名称	21
10. clear()	20	用法	21
名称	20	作用	22
用法	20	4. front()	22
作用	20	名称	22
二、stack（栈）	20	用法	22
1. stack 定义	20	作用	22
名称	20	5. back()	22
用法	20	名称	22
作用	20	用法	22
2. push()	20	作用	22
名称	20	6. size()	22
用法	20	名称	22
作用	20	用法	22

作用	22	作用	24
7. empty()	22	2. insert()	24
名称	22	名称	24
用法	22	用法	24
作用	22	作用	24
四、priority_queue（优先队列）	22	3. erase()	24
1. priority_queue 定义	22	名称	24
名称	22	用法 1: 删除指定元素	24
用法	22	用法 2: 删除迭代器位置	24
作用	22	用法 3: 删除区间	24
2. push()	22	作用	24
名称	22	4. find()	24
用法	22	名称	24
3. top()	22	用法	24
名称	22	作用	24
用法	22	5. count()	24
作用	23	名称	24
4. pop()	23	用法	24
名称	23	作用	25
用法	23	6. size()	25
5. empty()	23	名称	25
名称	23	用法	25
用法	23	作用	25
作用	23	7. empty()	25
6. size()	23	名称	25
名称	23	用法	25
用法	23	作用	25
作用	23	8. begin()	25
7. 小根堆	23	名称	25
名称	23	用法	25
用法	23	9. end()	25
作用	23	名称	25
priority_queue 自定义排序	23	用法	25
五、四种容器总结（考试记忆）	23	10. lower_bound()	25
第五章 set / multiset / map / unordered_map（STL 核心）	23	名称	25
一、set（集合）	23	用法	25
1. set 定义	23	作用	25
名称	23	11. upper_bound()	25
用法	24	名称	25
		用法	25
		作用	25

set 遍历	25	7. size()	27
方法 1: 范围 for	25	用法	27
方法 2: 迭代器	25	8. 遍历 map (重点)	27
set 常用场景	26	名称	27
去重排序	26	用法	27
二、multiset (可重复集合)	26	9. map 自动排序	28
1. multiset 定义	26	map 经典用途	28
名称	26	统计次数	28
用法	26	四、unordered_map (哈希表)	28
作用	26	1. unordered_map 定义	28
2. insert()	26	名称	28
用法	26	用法	28
3. erase()	26	作用	28
删除一个	26	2. 插入	28
删除全部	26	3. 查找	28
作用	26	4. 统计次数	28
4. count()	26	map 和 unordered_map 选择	28
用法	26	第六章 pair / tuple / iterator (迭代器)	29
作用	26	一、pair (存两个数据)	29
三、map (映射)	26	1. pair 定义	29
1. map 定义	26	名称	29
名称	26	用法	29
用法	26	作用	29
作用	26	2. first	29
2. 插入元素	27	名称	29
方法 1: 下标	27	用法	29
方法 2: insert	27	作用	29
作用	27	3. second	29
3. 访问元素	27	名称	29
名称	27	用法	29
用法	27	作用	29
4. erase()	27	4. make_pair()	29
用法	27	名称	29
作用	27	用法	29
5. find()	27	作用	29
用法	27	5. pair 比较	29
作用	27	作用	30
6. count()	27		
用法	27		

6. pair 排序	30	10. const_iterator	32
7. 自定义 pair 排序	30	名称	32
二、tuple（三个及以上数据）	30	用法	32
1. tuple 定义	30	作用	32
名称	30	11. reverse_iterator	32
用法	30	名称	32
作用	30	用法	33
2. get()	30	作用	33
名称	30	四、auto 关键字（STL 必用）	33
用法	30	名称	33
作用	31	用法	33
三、iterator（迭代器）	31	作用	33
1. iterator 定义	31	五、范围 for 循环（C++11）	33
名称	31	名称	33
用法	31	用法	33
作用	31	作用	33
2. begin()	31	六、STL 常用头文件总表（考试）	33
名称	31	第七章 C++ STL 上机考试万能模板	33
用法	31	1. 万能头文件模板	33
作用	31	名称	33
3. end()	31	用法	33
名称	31	作用	33
用法	31	2. 快速输入输出	34
作用	31	名称	34
4. 解引用 *	31	用法	34
名称	31	作用	34
用法	31	3. vector 输入模板	34
作用	31	场景	34
5. iterator 移动	32	写法	34
名称	32	作用	34
用法	32	4. vector 遍历模板	34
6. iterator 遍历 vector	32	方法 1：下标	34
方法	32	方法 2：范围 for	34
7. iterator 遍历 set	32	方法 3：引用	34
8. iterator 遍历 map（重点）	32	5. 数组排序模板	34
9. 箭头符号 ->	32	升序	34
名称	32	vector 排序	34
用法	32	降序	34
作用	32	6. 字符串排序模板	35
		名称	35

用法	35	23. pair 排序模板	37
7. vector 求最大最小值	35	默认	37
最大值	35	按第二关键字	38
最小值	35	24. 常用字符串处理组合	38
最大值位置	35	删除某字符	38
8. vector 求和模板	35	字符串反转	38
名称	35	字符串转数字	38
用法	35	数字转字符串	38
9. 去重模板	35	25. STL 复杂度速记	38
场景	35	26. 考试最常用 TOP20 函数（建议背）	38
写法	35	第八章 STL 补充知识（容易遗漏但考试可能出现）	38
10. 查找元素模板	35	一、 array （固定大小数组）	38
方法 1: find	35	1. array 定义	39
方法 2: count	35	名称	39
11. set 去重模板	35	用法	39
场景	35	作用	39
12. map 统计次数模板（超级高频）	36	2. size()	39
场景	36	名称	39
13. unordered_map 统计模板	36	用法	39
14. map 遍历模板	36	作用	39
方法 1	36	3. front()	39
方法 2（修改）	36	用法	39
15. stack 括号匹配模板	36	4. back()	39
场景	36	用法	39
16. queue BFS 模板	36	5. fill()	39
场景	36	用法	39
17. priority_queue 最大值模板	37	作用	39
默认大根堆	37	6. begin()	39
18. priority_queue 小根堆	37	用法	39
19. 二分查找模板	37	7. end()	39
lower_bound	37	用法	39
upper_bound	37	二、 unordered_set （哈希集合）	39
20. 全排列模板	37	1. unordered_set 定义	39
next_permutation	37	名称	39
21. 二维 vector 模板	37	用法	39
定义	37	作用	40
22. 二维 vector 遍历	37	2. insert()	40
		用法	40

3. find()	40	3. inner_product()	42
用法	40	名称	42
4. count()	40	用法	42
用法	40	作用	42
5. erase()	40	4. partial_sum()	42
用法	40	名称	42
6. size()	40	用法	42
用法	40	作用	42
三、 bitset （二进制位操作）	40	5. adjacent_difference()	42
1. bitset 定义	40	名称	42
名称	40	用法	42
用法	40	作用	42
作用	40	五、 functional （函数对象）	42
2. count()	40	1. greater()	42
用法	40	名称	42
作用	40	用法	42
3. size()	40	2. less()	42
用法	40	3. plus()	43
4. test()	41	用法	43
名称	41	4. minus()	43
用法	41	5. multiplies()	43
5. set()	41	6. divides()	43
用法	41	六、 lambda 表达式（非常重要）	43
6. reset()	41	1. 基本格式	43
用法	41	2. sort 中使用（最常见）	43
7. flip()	41	3. 降序	43
用法	41	4. pair 排序	43
8. to_string()	41	5. 自定义规则	43
用法	41	七、 STL 常见坑	43
9. to_ulong()	41	1. vector 删除元素	43
用法	41	2. vector 访问越界	43
四、 numeric 头文件其他函数	41	3. map 中 [] 自动创建	44
1. accumulate()	41	4. sort 区间	44
2. iota()	41	5. unique 必须配 erase	44
名称	41		
用法	41		
作用	41		

6. lower_bound 必须有序	44	13. capacity()	46
7. priority_queue 不能遍历	44	完整写法	46
第九章 C++ STL 函数速查表（机考打印版）	44	14. reserve()	46
一、vector 速查表（重点）	44	完整写法	46
1. push_back()	44	15. shrink_to_fit()	46
完整写法	44	完整写法	46
参数	44	16. at()	46
作用	44	完整写法	46
2. pop_back()	44	二、string 速查表（重点）	46
完整写法	44	1. size()	46
参数	44	2. length()	46
作用	44	3. empty()	47
3. size()	44	4. clear()	47
完整写法	44	5. push_back()	47
参数	45	6. pop_back()	47
返回	45	7. append()	47
4. empty()	45	写法	47
完整写法	45	8. substr()	47
参数	45	写法	47
返回	45	9. find()	47
5. clear()	45	写法	47
完整写法	45	10. replace()	47
6. front()	45	写法	47
完整写法	45	11. erase()	47
7. back()	45	删除位置	47
完整写法	45	12. insert()	47
8. begin()	45	写法	47
完整写法	45	13. compare()	48
9. end()	45	写法	48
完整写法	45	14. getline()	48
10. insert()	45	写法	48
完整写法	45	三、algorithm 速查表	48
11. erase()	45	1. sort()	48
完整写法 1	45	写法	48
完整写法 2	46		
12. resize()	46		
完整写法	46		

2. reverse()	48	八、priority_queue 速查表	50
写法	48	九、set 速查表	51
3. max()	48	十、考试最常忘参数总结	51
写法	48	第十章 C++ 常用头文件速查 (cmath + 其他高频库)	51
4. min()	48	一、cmath (数学函数库)【必背】	51
5. max_element()	48	1. abs()	51
写法	48	名称	51
6. min_element()	48	用法	51
7. find()	48	作用	51
写法	48	2. fabs()	52
8. count()	49	名称	52
9. fill()	49	用法	52
10. replace()	49	作用	52
11. unique()	49	3. sqrt()	52
12. lower_bound()	49	名称	52
13. upper_bound()	49	用法	52
14. binary_search()	49	作用	52
15. next_permutation()	49	4. pow()	52
四、numeric 速查表	49	名称	52
1. accumulate()	49	用法	52
写法	49	作用	52
2. iota()	49	5. exp()	52
3. partial_sum()	50	名称	52
五、map 速查表	50	用法	52
1. []	50	作用	53
2. insert()	50	6. log()	52
3. erase()	50	名称	52
4. find()	50	用法	52
5. count()	50	作用	53
6. 遍历	50	7. log10()	53
六、stack 速查表	50	名称	53
七、queue 速查表	50	用法	53
		作用	53
		8. sin()	53
		名称	53
		用法	53
		作用	53
		9. cos()	53
		用法	53

10. tan()	53	2. fixed	55
用法	53	用法	55
11. asin()	53	3. setprecision + fixed	55
名称	53	4. setw()	55
12. acos()	53	名称	55
13. atan()	53	用法	55
14. atan2()	53	作用	55
用法	53	5. left	55
作用	53	6. right	55
15. ceil()	53	7. setfill()	55
名称	53	用法	55
用法	54	三、cstring (C 风格字符串)	56
作用	54	1. strlen()	56
16. floor()	54	名称	56
名称	54	用法	56
用法	54	作用	56
作用	54	2. strcpy()	56
17. round()	54	用法	56
名称	54	3. strcat()	56
用法	54	用法	56
作用	54	4. strcmp()	56
18. trunc()	54	用法	56
名称	54	5. memset()	56
用法	54	用法	56
作用	54	四、cstdlib (常用工具)	56
19. fmod()	54	1. rand()	56
名称	54	用法	56
用法	54	2. srand()	56
作用	54	用法	56
20. hypot()	54	3. atoi()	57
名称	54	用法	57
用法	54	4. exit()	57
作用	55	用法	57
cmath 常量	55	五、sstream (字符串流)	57
M_PI	55	1. stringstream	57
二、iomanip (格式化输出)【重要】	55	定义	57
1. setprecision()	55		
名称	55		
用法	55		

字符串转数字	57	to_string()	59
数字转字符串	57	三、字符处理 (ctype)	59
作用	57	1. isdigit()	59
六、limits (极值)	57	名称	59
1. INT_MAX	57	用法	59
用法	57	2. isalpha()	59
2. INT_MIN	57	3. islower()	59
3. numeric_limits	57	4. isupper()	59
用法	57	5. tolower()	59
七、常用头文件总表 (考试版)	57	6. toupper()	60
第十一章 C++ 常用语法速查 (机考必备)	58	四、ASCII 码 (高频)	60
一、输入输出 (iostream)	58	1. 字符转数字	60
1. cin 输入	58	2. 数字转字符	60
用法	58	3. 字母大小写转换	60
2. 多个变量输入	58	五、long long 使用 (非常重要)	60
3. cout 输出	58	1. int 范围	60
用法	58	2. long long 范围	60
4. 输出多个内容	58	3. 定义	60
5. 换行	58	4. 常量	60
方法 1	58	六、三目运算符	60
方法 2 (推荐)	58	名称	60
6. getline()	58	格式	60
用法	58	七、位运算 (了解)	60
注意	58	1. 按位与 &	61
二、类型转换 (非常重要)	58	判断奇偶【常用】	61
1. int 转 double	58	2. 按位或	61
2. 强制类型转换	59	3. 异或 ^	61
写法 1	59	特点	61
写法 2 (C++ 推荐)	59	4. 左移 «	61
3. string 转 int	59	5. 右移 »	61
stoi()	59	八、常用数学技巧	61
4. string 转 long long	59	1. 最大公约数 gcd	61
5. string 转 double	59		
6. int 转 string	59		

2. 最小公倍数 lcm	61
3. 判断素数模板	61
九、 swap 交换	61
名称	61
用法	62
十、 auto 自动类型推导	62
用法	62
十一、引用 & (STL 常用)	62
普通变量	62
修改 vector 元素	62
十二、 const (防止修改)	62
十三、宏定义 (了解)	62
1. 常量	62
2. 宏函数	62
十四、结构体 struct	62
十五、自定义排序 (结构体)	63
十六、万能 main 模板 (考试推荐)	63
十七、考试最常用组合	63
1. 输入数组排序求和	63
2. 统计字符次数	63
3. 删除重复元素	63
4. 保留两位小数	63
5. 最大最小值	63
最终考试头文件优先级	63

明白你的需求。你要的是上机考试速查手册，不是教材式讲解，所以我会按照：

- 名称（函数/知识点）
- 用法（参数怎么写 + 示例代码）
- 作用（考试时知道什么时候用）

来整理。

由于内容非常庞大（尤其 `vector` + `string` + `algorithm` + `map/set/queue/stack`），一次回复放不下完整版本。我会分多次连续发送。

整体目录如下：

C++ STL 上机考试速查大全

第一部分：vector（极详细）

- vector 定义
- 初始化方式
- 遍历方式
- 所有成员函数
 - size()
 - capacity()
 - empty()
 - resize()
 - reserve()
 - push_back()
 - emplace_back()
 - pop_back()
 - insert()
 - erase()
 - clear()
 - front()
 - back()
 - at()
 - swap()
 - assign()
 - data()
 - begin()
 - end()
 - rbegin()
 - rend()

第二部分：string（极详细）

- 创建
- 输入输出
- 拼接
- 查找
- 截取
- 替换
- 删除
- 插入
- 字符转换
- 所有常用函数

第三部分：algorithm

- sort
- reverse
- max/min
- swap
- find
- count
- accumulate
- fill
- replace

- unique
- lower_bound
- upper_bound
- binary_search
- next_permutation
- prev_permutation
- rotate
- partition

第四部分：deque

第五部分：stack

第六部分：queue

第七部分：priority_queue

第八部分：set / multiset

第九部分：map / unordered_map

第十部分：pair

第十一部分：迭代器 iterator

第十二部分：常用万能模板

下面开始。

第一章 vector（重点）

头文件：

```
#include <vector>
using namespace std;
```

1. vector 定义

名称

```
vector
```

用法

```
vector<int> v;
```

定义一个存放 int 的动态数组。

其他：

```
vector<double> v1;
vector<string> v2;
vector<char> v3;
```

二维：

```
vector<vector<int>> matrix;
```

相当于二维数组。

作用

vector 是 STL 中最常用的动态数组。

和普通数组区别：

普通数组：

```
int a[100];
```

大小固定。

vector：

```
vector<int> v;
```

可以自动扩容。

2. vector 初始化

名称

初始化

用法

空 **vector**

```
vector<int> v;
```

创建:

```
{}
```

指定数量

```
vector<int> v(5);
```

结果:

```
0 0 0 0 0
```

指定数量和值

```
vector<int> v(5,10);
```

结果:

```
10 10 10 10 10
```

数组初始化

```
int a[]={1,2,3};
vector<int> v(a,a+3);
```

结果:

```
1 2 3
```

直接赋值

```
vector<int> v={1,2,3,4};
```

结果:

```
1 2 3 4
```

作用

创建 vector。

3. push_back()

名称

```
push_back()
```

用法

格式:

```
vector 名.push_back(元素);
```

例如:

```
vector<int> v;
v.push_back(10);
v.push_back(20);
```

结果:

```
10 20
```

作用

在 vector 末尾添加元素。
考试最常用。

4. pop_back()

名称

```
pop_back()
```

用法

```
v.pop_back();
```

例如:

```
vector<int> v={1,2,3};
v.pop_back();
```

结果:

```
1 2
```

作用

删除最后一个元素。
注意:
不能指定删除哪个。
只能删除尾部。

5. size()

名称

```
size()
```

用法

```
v.size();
```

例如:

```
vector<int> v={1,2,3};
cout<<v.size();
```

输出:

```
3
```

作用

返回 vector 当前元素数量。
注意:
不是容量。
例如:

```
vector<int> v(100);
```

size:

```
100
```

6. capacity()

名称

capacity()

用法

```
v.capacity();
```

例如:

```
vector<int> v;
v.push_back(1);
cout<<v.capacity();
```

可能输出:

1

或者:

2

作用

查看 vector 当前申请的内存空间。

区别:

函数	作用
size()	已有多少元素
capacity()	最多能放多少元素

考试一般用 size。

7. empty()

名称

empty()

用法

```
v.empty();
```

例如:

```
vector<int> v;
if(v.empty())
{
    cout<<"空";
}
```

返回:

true

作用

判断 vector 是否为空。

等价:

```
v.size()==0
```

8. clear()

名称

clear()

用法

```
v.clear();
```

例如:

```
vector<int> v={1,2,3};
v.clear();
```

结果:

```
{}
```

作用

删除所有元素。

注意:

不会释放空间。

例如:

```
v.clear();
v.capacity();
```

容量可能仍然存在。

9. front()

名称

front()

用法

```
v.front();
```

例如:

```
vector<int> v={10,20,30};
cout<<v.front();
```

输出:

10

作用

获取第一个元素。

等价:

```
v[0]
```

10. back()

名称

back()

用法

```
v.back();
```

例如:

```
vector<int> v={10,20,30};
cout<<v.back();
```

输出:

30

作用

获取最后一个元素。
等价：

```
v[v.size()-1]
```

11. 下标访问 []

名称

```
operator []
```

用法

```
v[index];
```

例如：

```
vector<int> v={5,6,7};  
cout<<v[1];
```

输出：

```
6
```

作用

访问指定位置元素。
注意：
不检查越界。

12. at()

名称

```
at()
```

用法

```
v.at(index);
```

例如：

```
vector<int> v={1,2,3};  
cout<<v.at(1);
```

输出：

```
2
```

作用

访问元素。

区别：

```
v[10];
```

越界可能崩溃。

```
v.at(10);
```

会主动报错。

13. begin()

名称

```
begin()
```

用法

```
v.begin();
```

例如：

```
vector<int> v={1,2,3};  
cout<<*v.begin();
```

输出：

```
1
```

作用

返回第一个元素的迭代器。

常用于：

```
sort(v.begin(),v.end());
```

14. end()

名称

```
end()
```

用法

```
v.end();
```

例如：

```
sort(v.begin(),v.end());
```

作用

返回最后一个元素后面的位置。

注意：
不是最后一个元素。

最后元素：

```
*(v.end()-1)
```

15. rbegin()

名称

```
rbegin()
```

用法

```
v.rbegin();
```

例如：

```
vector<int> v={1,2,3};  
cout<<*v.rbegin();
```

输出：

```
3
```

作用

反向迭代器开始位置。

16. rend()

名称

```
rend()
```

用法

```
v.rend();
```

例如:

```
for(auto i=v.rbegin();i!=v.rend();i++)
{
    cout<<*i;
}
```

输出:

```
321
```

作用

反向遍历结束位置。

17. insert()

名称

```
insert()
```

用法

格式:

```
v.insert(位置,元素);
```

例如:

```
vector<int> v={1,3};
v.insert(v.begin()+1,2);
```

结果:

```
1 2 3
```

插入多个:

```
v.insert(v.begin(),3,10);
```

结果:

```
10 10 10 ...
```

作用

在指定位置插入元素。

18. erase()

名称

```
erase()
```

用法

删除一个:

```
v.erase(v.begin()+2);
```

例如:

```
vector<int> v={1,2,3};
v.erase(v.begin()+1);
```

结果:

```
1 3
```

删除区间:

```
v.erase(v.begin(),v.begin()+2);
```

删除:

前两个元素

作用

删除指定位置或者范围元素。

19. resize()

名称

```
resize()
```

用法

改变大小:

```
v.resize(10);
```

不足补 0。

例如:

```
vector<int> v={1,2};
v.resize(5);
```

结果:

```
1 2 0 0 0
```

指定值:

```
v.resize(5,100);
```

结果:

```
1 2 100 100 100
```

作用

改变 vector 元素数量。

20. reserve()

名称

```
reserve()
```

用法

```
v.reserve(100);
```

作用

提前申请空间。

区别:

resize: 改变元素数量。

reserve: 只改变容量。

例如:

```
vector<int> v;
v.reserve(100);
cout<<v.size();
```

输出:

```
0
```

21. swap()

名称

```
swap()
```

用法

两个 vector 交换：

```
v1.swap(v2);
```

例如：

```
vector<int> a={1,2};
vector<int> b={3,4};
a.swap(b);
```

结果：

```
a:
3 4
```

```
b:
1 2
```

作用

交换两个 vector 内容。

22. assign()

名称

```
assign()
```

用法

数量和值：

```
v.assign(5,10);
```

结果：

```
10 10 10 10 10
```

区间：

```
v.assign(v2.begin(),v2.end());
```

作用

重新给 vector 赋值。

23. data()

名称

```
data()
```

用法

```
int *p=v.data();
```

例如：

```
vector<int> v={1,2,3};
int *p=v.data();
cout<<p[0];
```

输出：

```
1
```

作用

返回底层数组指针。
用于和 C 语言数组交互。

vector 遍历（考试必背）

方法 1：下标

```
for(int i=0;i<v.size();i++)
{
    cout<<v[i];
}
```

方法 2：范围 for

```
for(auto x:v)
{
    cout<<x;
}
```

方法 3：迭代器

```
for(auto it=v.begin();it!=v.end();it++)
{
    cout<<*it;
}
```

方法 4：反向

```
for(auto it=v.rbegin();it!=v.rend();it++)
{
    cout<<*it;
}
```

第二章 string（重点）

头文件：

```
#include <string>
using namespace std;
```

string 是 STL 中处理字符串的容器。
相比 C 语言的传统字符数组：

```
char s[100];
```

使用 string：

```
string s;
```

优点是 ** 不用考虑长度，不需要手动管理结束标记 '\0' 。

1. string 定义

名称

```
string
```

用法

普通定义：

```
string s;
```

初始化:

```
string s="hello";
```

定义多个:

```
string a="abc";  
string b="def";
```

作用

创建字符串对象。

2. string 输入

方法 1: cin（输入一个单词）

名称

```
cin
```

用法

```
string s;  
cin>>s;
```

若输入:

```
hello
```

结果:

```
s="hello"
```

注意：遇到空格、换行即停止读取。

方法 2: getline（输入整行）

名称

```
getline()
```

用法

```
getline(cin,s);
```

示例:

```
string s;  
getline(cin,s);  
cout<<s;
```

若输入:

```
hello world
```

输出:

```
hello world
```

作用

读取包含空格的一整行字符串，** 上机考试非常常用 **。

3. size()

名称

```
size()
```

用法

```
s.size();
```

示例:

```
string s="hello";
```

```
cout<<s.size();
```

输出结果:

```
5
```

作用

获取字符串当前的实际长度。

4. length()

名称

```
length()
```

用法

```
s.length();
```

示例:

```
string s="abc";  
cout<<s.length();
```

输出结果:

```
3
```

作用

获取字符串长度。
注意：功能与 **size()** 完全一样，机考用哪个都可以。

5. empty()

名称

```
empty()
```

用法

```
s.empty();
```

示例:

```
string s="";  
if(s.empty())  
{  
    cout<<"空";  
}
```

作用

判断字符串是否为空（长度是否为 0）。

6. 字符访问 []

名称

```
operator []
```

用法

```
格式:  
s[index];
```

例如:

```
string s="hello";  
cout<<s[0];
```

输出：

```
h
```

修改：

```
s[0]='H';
```

结果：

```
Hello
```

作用

访问或者修改指定位置字符。

注意：

下标从 **0** 开始。

7. at()

名称

```
at()
```

用法

```
s.at(index);
```

例如：

```
string s="abc";
cout<<s.at(1);
```

输出：

```
b
```

修改：

```
s.at(1)='B';
```

结果：

```
aBc
```

作用

访问指定位置字符。

区别：

```
s[100];
```

越界不检查。

```
s.at(100);
```

会抛异常。

8. front()

名称

```
front()
```

用法

```
s.front();
```

例如：

```
string s="hello";
cout<<s.front();
```

输出：

```
h
```

作用

获取第一个字符。

等价：

```
s[0]
```

9. back()

名称

```
back()
```

用法

```
s.back();
```

例如：

```
string s="hello";
cout<<s.back();
```

输出：

```
o
```

作用

获取最后一个字符。

等价：

```
s[s.size()-1]
```

10. push_back()

名称

```
push_back()
```

用法

格式：

```
s.push_back(字符);
```

例如：

```
string s="abc";
s.push_back('d');
```

结果：

```
abcd
```

注意：

只能添加一个字符。

错误：

```
s.push_back("hello");
```

正确：

```
s+="hello";
```

作用

在字符串末尾添加字符。

11. pop_back()

名称

```
pop_back()
```

用法

```
s.pop_back();
```

例如:

```
string s="hello";
s.pop_back();
```

结果:

```
hell
```

作用

删除最后一个字符。

12. append()

名称

```
append()
```

用法

添加字符串:

```
s.append("abc");
```

例如:

```
string s="hello";
s.append(" world");
```

结果:

```
hello world
```

也可以:

```
s.append(t);
```

其中:

```
string t="abc";
```

作用

在末尾追加字符串。

等价:

```
s+="abc";
```

添加字符:

```
s+='!';
```

结果:

```
hello!
```

作用

连接两个字符串。

14. insert()

名称

```
insert()
```

用法

格式:

```
s.insert(位置, 字符串);
```

例如:

```
string s="abcd";
s.insert(2, "XX");
```

过程:

```
ab + XX + cd
```

结果:

```
abXXcd
```

插入一个字符:

```
s.insert(0,1,'A');
```

结果:

```
Aabcd
```

作用

在指定位置插入内容。

15. erase()

名称

```
erase()
```

用法 **1**: 删除指定位置

格式:

```
s.erase(pos);
```

例如:

```
string s="abcdef";
s.erase(2);
```

结果:

```
ab
```

从位置 2 开始全部删除。

13. 字符串连接 +

名称

字符串拼接

用法

```
s1+s2;
```

例如:

```
string a="hello";
string b="world";
string c=a+b;
```

结果:

```
helloworld
```

用法 2：删除指定长度

格式：

```
s.erase(pos, len);
```

例如：

```
string s="abcdef";
s.erase(2,3);
```

删除：

```
cde
```

结果：

```
abf
```

作用

删除字符串中的字符。

16. clear()

名称

```
clear()
```

用法

```
s.clear();
```

例如：

```
string s="hello";
s.clear();
```

结果：

```
""
```

作用

清空字符串。

17. substr()

名称

```
substr()
```

用法

格式：

```
s.substr(开始位置, 长度);
```

例如：

```
string s="abcdef";
string t=s.substr(2,3);
```

从：

```
c 开始
取3个
```

结果：

```
cde
```

省略长度：

```
s.substr(2);
```

结果：

```
cdef
```

作用

截取字符串。
考试非常常用。

18. find()

名称

```
find()
```

用法

格式：

```
s.find(查找内容);
```

例如：

```
string s="hello";
cout<<s.find("ll");
```

输出：

```
2
```

因为：

```
h e l l o
0 1 2 3 4
```

查找字符：

```
s.find('o');
```

输出：

```
4
```

不存在：

```
s.find("xx");
```

返回：

```
string::npos
```

例如：

```
if(s.find("abc")!=string::npos)
{
    cout<<"存在";
}
```

作用

查找子串或者字符第一次出现的位置。

19. rfind()

名称

```
rfind()
```

用法

```
s.rfind("abc");
```

例如：

```
string s="abcabc";
cout<<s.rfind("abc");
```

输出：

```
3
```

作用

从后往前查找。
返回最后一次出现位置。

20. replace()

名称

```
replace()
```

用法

格式：

```
s.replace(位置,长度,替换内容);
```

例如：

```
string s="abcdef";
s.replace(2,3,"XYZ");
```

原：

```
ab cde f
```

替换：

```
ab XYZ f
```

结果：

```
abXYZf
```

作用

替换字符串中的部分内容。

21. compare()

名称

```
compare()
```

用法

```
s1.compare(s2);
```

例如：

```
string a="abc";
string b="abc";
cout<<a.compare(b);
```

输出：

```
0
```

结果：

作用

比较字符串。

返回值	意义
0	相等
<0	a 小于 b
>0	a 大于 b

22. begin()

名称

```
begin()
```

用法

```
s.begin();
```

例如：

```
cout<<*s.begin();
```

输出：

```
第一个字符
```

作用

返回字符串第一个字符迭代器。

常用于：

```
sort(s.begin(),s.end());
```

23. end()

名称

```
end()
```

用法

```
s.end();
```

例如：

```
sort(s.begin(),s.end());
```

作用

返回最后一个字符后的位置。

24. rbegin()

名称

```
rbegin()
```

用法

```
s.rbegin();
```

例如：

```
cout<<*s.rbegin();
```

输出：

```
最后一个字符
```

作用

反向遍历开始。

25. rend()

名称

```
rend()
```

用法

```
s.rend();
```

例如:

```
for(auto it=s.rbegin();it!=s.rend();it++)
{
    cout<<*it;
}
```

字符串:

```
abc
```

输出:

```
cba
```

作用

反向遍历结束位置。

26. stoi()

名称

```
stoi()
```

用法

```
stoi(字符串);
```

例如:

```
string s="123";
int x=stoi(s);
```

结果:

```
x=123
```

作用

字符串转整数。

27. stoll()

名称

```
stoll()
```

用法

```
long long x=stoll(s);
```

例如:

```
string s="123456789";
long long x=stoll(s);
```

作用

字符串转 long long。

28. stod()

名称

```
stod()
```

用法

```
double x=stod(s);
```

例如:

```
string s="3.14";
double x=stod(s);
```

作用

字符串转 double。

29. to_string()

名称

```
to_string()
```

用法

```
to_string(数字);
```

例如:

```
int x=123;
string s=to_string(x);
```

结果:

```
"123"
```

作用

数字转字符串。

30. sort 字符串

名称

```
sort()
```

用法

```
sort(s.begin(),s.end());
```

例如:

```
string s="dcba";
sort(s.begin(),s.end());
```

结果:

```
abcd
```

作用

按照 ASCII 排序字符。

31. reverse 字符串

名称

`reverse()`

用法

```
reverse(s.begin(),s.end());
```

例如:

```
string s="hello";
reverse(s.begin(),s.end());
```

结果:

olleh

作用

反转字符串。

string 考试高频组合

1. 判断回文字符串

```
string s;
string t=s;
reverse(t.begin(),t.end());

if(s==t)
    cout<<"yes";
```

作用:

判断正反是否一样。

2. 统计字符数量

```
int cnt[26]={0};
for(char c:s)
{
    cnt[c-'a']++;
}
```

作用:

统计每个字母出现次数。

3. 删除空格

```
s.erase(remove(s.begin(),s.end(),' '),s.end());
```

作用:

删除所有空格。

(这里涉及 algorithm 的 remove, 后面详细讲)

第三章 **algorithm** (STL 算法库, 考试核心)

头文件:

```
#include <algorithm>
```

部分数学算法需要:

```
#include <numeric>
```

1. **sort()**

名称

`sort()`

用法

基本格式:

```
sort(开始位置,结束位置);
```

例如 vector:

```
vector<int> v={3,1,5,2};
sort(v.begin(),v.end());
```

结果:

1 2 3 5

数组:

```
int a[]={3,1,5,2};
sort(a,a+4);
```

结果:

1 2 3 5

降序:

格式:

```
sort(begin,end,greater<类型>());
```

例如:

```
sort(v.begin(),v.end(),greater<int>());
```

结果:

5 3 2 1

自定义排序:

格式:

```
sort(begin,end,比较函数);
```

例如:

```
bool cmp(int a,int b)
{
    return a>b;
}
```

```
sort(v.begin(),v.end(),cmp);
```

作用

排序。

考试最常用函数之一。

默认:

升序

2. **stable_sort()**

名称

`stable_sort()`

用法

和 `sort` 一样：

```
stable_sort(v.begin(), v.end());
```

作用

稳定排序。

区别：

sort:
相同元素顺序可能改变。
stable_sort:
保证相同元素原来的顺序。

考试：
普通排序直接 `sort`。

3. reverse()

名称

```
reverse()
```

用法

格式：

```
reverse(开始, 结束);
```

例如：

```
vector<int> v={1,2,3,4};
reverse(v.begin(), v.end());
```

结果：

```
4 3 2 1
```

字符串：

```
string s="hello";
reverse(s.begin(), s.end());
```

结果：

```
olleh
```

作用

反转顺序。

4. rotate()

名称

```
rotate()
```

用法

格式：

```
rotate(first, middle, last);
```

例如：

```
vector<int> v={1,2,3,4,5};
```

```
rotate(v.begin(), v.begin()+2, v.end());
```

过程：
以第 3 个元素为开头：

```
3 4 5 1 2
```

作用

旋转数组。

例如：

左旋：

```
1 2 3 4 5
```

左移 2 位

```
3 4 5 1 2
```

5. swap()

名称

```
swap()
```

用法

```
swap(a, b);
```

例如：

```
int a=10;
int b=20;
swap(a, b);
```

结果：

```
a=20
b=10
```

字符串：

```
swap(s1, s2);
```

作用

交换两个变量。

6. max()

名称

```
max()
```

用法

两个数：

```
max(a, b);
```

例如：

```
cout<<max(3, 5);
```

输出：

5

多个:

```
max({1,5,3});
```

输出:

5

作用

返回最大值。

7. min()

名称

```
min()
```

用法

```
min(a,b);
```

例如:

```
cout<<min(3,5);
```

输出:

3

多个:

```
min({3,1,5});
```

输出:

1

作用

返回最小值。

8. max_element()

名称

```
max_element()
```

用法

格式:

```
max_element(begin,end);
```

注意:

返回的是迭代器。

例如:

```
vector<int> v={3,7,2};
```

```
auto it=max_element(v.begin(),v.end());
```

```
cout<<*it;
```

输出:

7

作用

找容器最大元素位置。

如果想得到下标:

```
int pos=max_element(v.begin(),v.end())-v.begin();
```

9. min_element()

名称

```
min_element()
```

用法

```
auto it=min_element(v.begin(),v.end());
```

例如:

```
cout<<*min_element(v.begin(),v.end());
```

输出:

2

作用

找最小元素。

10. find()

名称

```
find()
```

用法

格式:

```
find(begin,end,value);
```

例如:

```
vector<int> v={1,2,3,4};
```

```
auto it=find(v.begin(),v.end(),3);
```

找到:

3

输出:

```
cout<<*it;
```

不存在:

```
if(it==v.end())
```

```
{
```

```
    cout<<"不存在";
```

```
}
```

作用

查找元素。

注意：
返回迭代器。

计算：

0+1+2+3+4

结果：

10

11. count()

名称

count()

用法

格式：

count(begin,end,value);

例如：

```
vector<int> v={1,2,2,3};
int x=count(v.begin(),v.end(),2);
```

结果：

2

字符串：

```
string s="hello";
cout<<count(s.begin(),s.end(),'l');
```

输出：

2

作用

统计元素出现次数。

注意：

第三个参数类型决定返回类型。

例如：

```
vector<double> v={1.2,2.3};
double sum=accumulate(v.begin(),v.end(),0.0);
```

必须：

0.0

不能：

0

否则可能整数计算。

作用

求和。

13. fill()

名称

fill()

用法

格式：

fill(begin,end,value);

例如：

数组：

```
int a[5];
fill(a,a+5,10);
```

结果：

10 10 10 10 10

vector:

fill(v.begin(),v.end(),0);

作用

全部赋值。

14. replace()

名称

replace()

12. accumulate()

头文件：

#include <numeric>

名称

accumulate()

用法

格式：

accumulate(begin,end,初始值);

三个参数：

- 1. 开始位置
- 2. 结束位置
- 3. 初始值

例如：

```
vector<int> v={1,2,3,4};
int sum=accumulate(v.begin(),v.end(),0);
```

用法

格式:

```
replace(begin,end,旧值,新值);
```

例如:

```
vector<int> v={1,2,2,3};
replace(v.begin(),v.end(),2,99);
```

结果:

```
1 99 99 3
```

字符串:

```
string s="hello";
replace(s.begin(),s.end(),'l','x');
```

结果:

```
hexxo
```

作用

替换所有指定元素。

15. unique()

名称

```
unique()
```

用法

常和 sort 一起:

```
sort(v.begin(),v.end());
v.erase(unique(v.begin(),v.end()),v.end());
```

例如:

原:

```
1 2 2 3 3 3
```

unique:

```
1 2 3 ? ? ?
```

erase 删除后:

```
1 2 3
```

作用

去除连续重复元素。

注意:

单独 unique 不会真正删除。

16. lower_bound()

名称

```
lower_bound()
```

用法

要求:

数组必须有序。

格式:

```
lower_bound(begin,end,value);
```

例如:

```
vector<int> v={1,3,5,7};
auto it=lower_bound(v.begin(),v.end(),5);
```

结果:

5 的位置

下标:

```
int pos=it-v.begin();
```

输出:

```
2
```

作用

寻找第一个:

```
>= value
```

的位置。

例如:

数组:

```
1 3 5 7
```

找 4:

返回:

```
5
```

17. upper_bound()

名称

```
upper_bound()
```

用法

```
upper_bound(v.begin(),v.end(),value);
```

例如:

```
vector<int> v={1,3,5,7};
auto it=upper_bound(v.begin(),v.end(),5);
```

返回:

```
7
```

作用

寻找第一个:

```
> value
```

的位置。

18. binary_search()

名称

```
binary_search()
```

用法

要求：
必须排序。

格式：

```
binary_search(begin,end,value);
```

例如：

```
vector<int> v={1,3,5,7};
bool flag=binary_search(v.begin(),v.end(),5);
```

结果：

```
true
```

作用

二分查找是否存在。

返回：

```
true/false
```

19. next_permutation()

名称

```
next_permutation()
```

用法

```
next_permutation(begin,end);
```

例如：

```
string s="abc";
next_permutation(s.begin(),s.end());
```

结果：

```
acb
```

连续：

```
while(next_permutation(s.begin(),s.end()))
{
    cout<<s;
}
```

输出所有排列。

作用

生成下一个字典序排列。

20. prev_permutation()

名称

```
prev_permutation()
```

用法

```
prev_permutation(s.begin(),s.end());
```

例如：

```
cba
```

变：

```
cab
```

作用

生成上一个排列。

algorithm 高频考试组合

1. vector 排序去重

```
sort(v.begin(),v.end());
v.erase(unique(v.begin(),v.end()),v.end());
```

作用：
去除重复元素。

2. 求最大最小值

```
int mx=*max_element(v.begin(),v.end());
int mn=*min_element(v.begin(),v.end());
```

3. 求和

```
int sum=accumulate(v.begin(),v.end(),0);
```

4. 判断存在

```
if(find(v.begin(),v.end(),x)!=v.end())
{
    cout<<"存在";
}
```

第四章 deque / stack / queue / priority_queue

一、deque（双端队列）

头文件：

```
#include <deque>
```

1. deque 定义

名称

```
deque
```

用法

```
deque<int> dq;
```

初始化:

```
deque<int> dq={1,2,3};
```

作用

双端队列。

特点:

- 两端都可以插入
- 两端都可以删除
- 支持随机访问

类似:

```
vector + queue
```

用法

```
dq.pop_back();
```

例如:

```
deque<int> dq={1,2,3};
dq.pop_back();
```

结果:

```
1 2
```

作用

删除最后元素。

2. push_back()

名称

```
push_back()
```

用法

```
dq.push_back(x);
```

例如:

```
deque<int> dq;
dq.push_back(10);
dq.push_back(20);
```

结果:

```
10 20
```

作用

尾部添加元素。

5. pop_front()

名称

```
pop_front()
```

用法

```
dq.pop_front();
```

例如:

```
deque<int> dq={1,2,3};
dq.pop_front();
```

结果:

```
2 3
```

作用

删除第一个元素。

3. push_front()

名称

```
push_front()
```

用法

```
dq.push_front(x);
```

例如:

```
deque<int> dq={2,3};
dq.push_front(1);
```

结果:

```
1 2 3
```

作用

头部插入元素。

6. front()

名称

```
front()
```

用法

```
dq.front();
```

例如:

```
cout<<dq.front();
```

作用

访问第一个元素。

7. back()

名称

```
back()
```

用法

```
dq.back();
```

4. pop_back()

名称

```
pop_back()
```

作用

访问最后元素。

8. size()

名称

```
size()
```

用法

```
dq.size();
```

作用

获取元素数量。

9. empty()

名称

```
empty()
```

用法

```
dq.empty();
```

作用

判断是否为空。

10. clear()

名称

```
clear()
```

用法

```
dq.clear();
```

作用

清空 deque。

二、stack（栈）

头文件：

```
#include <stack>
```

1. stack 定义

名称

```
stack
```

用法

```
stack<int> st;
```

作用

先进后出：

LIFO

类似：

一摞盘子。

最后放的最先拿。

2. push()

名称

```
push()
```

用法

```
st.push(x);
```

例如：

```
stack<int> st;
```

```
st.push(1);
```

```
st.push(2);
```

```
st.push(3);
```

栈：

3

2

1

作用

入栈。

3. pop()

名称

```
pop()
```

用法

```
st.pop();
```

例如：

```
st.pop();
```

删除：

3

剩：

2

1

作用

删除栈顶元素。

注意：

pop 没有返回值。

错误：

```
int x=st.pop();
```

错误。

4. top()

名称

```
top()
```

用法

```
st.top();
```

例如:

```
cout<<st.top();
```

作用

查看栈顶元素。

5. size()

名称

```
size()
```

用法

```
st.size();
```

作用

获取元素数量。

6. empty()

名称

```
empty()
```

用法

```
st.empty();
```

作用

判断栈是否为空。

stack 经典模板

括号匹配

```
string s;
stack<char> st;
for(char c:s)
{
    if(c=='(')
        st.push(c);

    else if(c==')')
    {
        if(st.empty())
            cout<<"NO";

        st.pop();
    }
}
```

作用:

判断括号是否合法。

三、queue（普通队列）

头文件:

```
#include <queue>
```

1. queue 定义

名称

```
queue
```

用法

```
queue<int> q;
```

作用

先进先出:

```
FIFO
```

类似排队:

先来先走。

2. push()

名称

```
push()
```

用法

```
q.push(x);
```

例如:

```
q.push(1);
q.push(2);
q.push(3);
```

队列:

```
1 2 3
```

作用

队尾加入元素。

3. pop()

名称

```
pop()
```

用法

```
q.pop();
```

例如:

原:

```
1 2 3
```

执行:

```
q.pop();
```

结果:

```
2 3
```

作用

删除队头元素。

注意：
没有返回值。

4. front()

名称

front()

用法

q.front();

例如：

cout<<q.front();

输出：
队头元素。

作用

访问队头。

5. back()

名称

back()

用法

q.back();

作用

访问队尾。

6. size()

名称

size()

用法

q.size();

作用

元素数量。

7. empty()

名称

empty()

用法

q.empty();

作用

判断队列为空。

四、priority_queue（优先队列）

头文件：

#include <queue>

1. priority_queue 定义

名称

priority_queue

用法

默认：

priority_queue<int> pq;

作用

自动排序的队列。

默认：

大根堆：

最大值永远在顶部

2. push()

名称

push()

用法

pq.push(x);

例如：

priority_queue<int> pq;

pq.push(3);
pq.push(10);
pq.push(5);

内部：

10
5
3

3. top()

名称

top()

用法

pq.top();

例如：

cout<<pq.top();

输出：

10

作用

查看最大元素。

4. pop()

名称

pop()

用法

pq.pop();

删除：
当前最大值。

5. empty()

名称

empty()

用法

pq.empty();

作用

判断为空。

6. size()

名称

size()

用法

pq.size();

作用

元素数量。

7. 小根堆

名称

小根 priority_queue

用法

priority_queue<int,vector<int>,greater<int>> pq;

例如:

pq.push(5);
pq.push(2);
pq.push(8);

top:

2

作用

让最小值在顶部。
考试非常重要。

priority_queue 自定义排序

例如：
结构体:

```
struct Node
{
    int x;
    int y;
};
```

比较:

```
struct cmp
{
    bool operator()(Node a,Node b)
    {
        return a.x>b.x;
    }
};
```

定义:

priority_queue<Node,vector<Node>,cmp> pq;

作用：
复杂排序。

五、四种容器总结（考试记忆）

容器	特点	常用函数
vector	动态数组	push_back erase sort
deque	两端操作	push_front
stack	后进先出	push pop top
queue	先进先出	push pop front
priority_queue	自动排序	push top pop

第五章 set / multiset / map / unordered_map（STL 核心）

这一章非常重要。

考试中常见：

- 去重
- 查找某个元素是否存在
- 统计次数
- 建立映射关系
- 字符频率统计

一、set（集合）

头文件:

#include <set>

1. set 定义

名称

set

用法

```
set<int> s;
```

初始化:

```
set<int> s={3,1,2,2};
```

结果:

```
1 2 3
```

特点:

- 1. 自动排序
- 2. 自动去重

作用

保存唯一元素。
类似数学集合。

2. insert()

名称

```
insert()
```

用法

```
s.insert(value);
```

例如:

```
set<int> s;  
  
s.insert(3);  
s.insert(1);  
s.insert(3);
```

结果:

```
1 3
```

作用

插入元素。
注意:
重复元素不会加入。

3. erase()

名称

```
erase()
```

用法 1: 删除指定元素

```
s.erase(value);
```

例如:

```
set<int> s={1,2,3};  
  
s.erase(2);
```

结果:

```
1 3
```

用法 2: 删除迭代器位置

```
auto it=s.begin();  
  
s.erase(it);
```

删除第一个元素。

用法 3: 删除区间

```
s.erase(s.begin(),s.end());
```

清空。

作用

删除元素。

4. find()

名称

```
find()
```

用法

```
s.find(value);
```

例如:

```
set<int> s={1,2,3};  
  
auto it=s.find(2);
```

如果存在:
返回位置。

不存在:

```
s.end()
```

判断:

```
if(s.find(2)!=s.end())  
{  
    cout<<"存在";  
}
```

作用

查找元素。

5. count()

名称

```
count()
```

用法

```
s.count(value);
```

例如:

```
set<int> s={1,2,3};  
  
cout<<s.count(2);
```

输出:

```
1
```

不存在:

```
0
```

作用

判断元素是否存在。
因为 `set` 没有重复：
返回只能：
`0` 或 `1`

6. size()

名称

`size()`

用法

`s.size();`

作用

元素数量。

7. empty()

名称

`empty()`

用法

`s.empty();`

作用

判断为空。

8. begin()

名称

`begin()`

用法

`auto it=s.begin();`

例如：

`cout<<*s.begin();`

输出：
最小元素。

9. end()

名称

`end()`

用法

`s.end();`

作用：
最后一个元素之后。

10. lower_bound()

名称

`lower_bound()`

用法

`auto it=s.lower_bound(x);`

例如：

`set<int> s={1,3,5,7};`
`auto it=s.lower_bound(4);`

返回：

`5`

作用

找第一个：

`>=x`

11. upper_bound()

名称

`upper_bound()`

用法

`auto it=s.upper_bound(x);`

例如：

`auto it=s.upper_bound(5);`

返回：

`7`

作用

找第一个：

`>x`

set 遍历

方法 1：范围 for

`for(auto x:s)`
{
 `cout<<x;`
}

输出：

`1 2 3`

方法 2：迭代器

`for(auto it=s.begin();it!=s.end();it++)`
{
 `cout<<*it;`
}

set 常用场景

去重排序

原：

```
vector<int> v={3,1,2,2};
```

方法：

```
set<int> s(v.begin(),v.end());
```

结果：

```
1 2 3
```

二、multiset（可重复集合）

头文件：

```
#include <set>
```

1. multiset 定义

名称

```
multiset
```

用法

```
multiset<int> ms;
```

例如：

```
multiset<int> ms={1,2,2,3};
```

结果：

```
1 2 2 3
```

作用

允许重复元素的 set。

区别：

	set	multiset
重复	不允许	允许
排序	自动	自动

2. insert()

用法

```
ms.insert(x);
```

例如：

```
ms.insert(2);  
ms.insert(2);
```

结果：

```
2 2
```

3. erase()

重点！

删除一个

```
auto it=ms.find(2);  
ms.erase(it);
```

删除一个 2。

删除全部

```
ms.erase(2);
```

所有 2 都会删除。

作用

删除元素。

4. count()

用法

```
ms.count(x);
```

例如：

```
multiset<int> ms={1,2,2,2,3};  
cout<<ms.count(2);
```

输出：

```
3
```

作用

统计出现次数。

三、map（映射）

头文件：

```
#include <map>
```

1. map 定义

名称

```
map
```

用法

```
map<string,int> mp;
```

表示：

```
字符串 -> 整数
```

例如：

```
mp["apple"]=5;
```

形成：

```
apple 5
```

作用

键值映射。

类似：
字典：

单词 → 解释
姓名 → 成绩

2. 插入元素

方法 1：下标

```
mp[key]=value;
```

例如：

```
map<string,int> mp;  
mp["Tom"]=90;
```

方法 2: insert

```
mp.insert(pair<string,int>("Tom",90));
```

或者：

```
mp.insert({"Tom",90});
```

作用

添加键值对。

3. 访问元素

名称

```
[]
```

用法

```
mp[key];
```

例如：

```
cout<<mp["Tom"];
```

输出：

```
90
```

注意：
如果 key 不存在：

```
mp["Bob"];
```

会自动创建：

```
Bob 0
```

4. erase()

用法

删除 key:

```
mp.erase(key);
```

例如：

```
mp.erase("Tom");
```

删除：

```
Tom
```

作用

删除映射。

5. find()

用法

```
mp.find(key);
```

例如：

```
if(mp.find("Tom")!=mp.end())  
{  
    cout<<"存在";  
}
```

作用

查找 key。

6. count()

用法

```
mp.count(key);
```

例如：

```
mp.count("Tom");
```

返回：

```
1
```

不存在：

```
0
```

7. size()

用法

```
mp.size();
```

作用：
键值对数量。

8. 遍历 map（重点）

名称

```
pair
```

用法

```
for(auto p:mp)  
{  
    cout<<p.first<<" "<<p.second;  
}
```

例如：

```
Tom 90  
Bob 80
```

输出:

```
Tom 90
Bob 80
```

其中:

```
p.first
```

表示:

key

```
p.second
```

表示:

value

9. map 自动排序

默认:

按照 key 升序。

例如:

```
map<int,string> mp;

mp[3]="c";
mp[1]="a";
mp[2]="b";
```

遍历:

```
1 a
2 b
3 c
```

map 经典用途

统计次数

例如:

统计字符串字符:

```
string s="hello";

map<char,int> mp;

for(char c:s)
{
    mp[c]++;
}
```

结果:

```
h 1
e 1
l 2
o 1
```

四、unordered_map (哈希表)

头文件:

```
#include <unordered_map>
```

1. unordered_map 定义

名称

```
unordered_map
```

用法

```
unordered_map<string,int> mp;
```

作用

类似 map。

区别:

	map	unordered_map
底层	红黑树	哈希表
排序	自动排序	无序
速度	O(logn)	平均 O(1)

2. 插入

```
mp[key]=value;
```

例如:

```
mp["apple"]=10;
```

3. 查找

```
mp.find(key);
```

例如:

```
if(mp.find("apple")!=mp.end())
{
    cout<<"yes";
}
```

4. 统计次数

最常用:

```
unordered_map<char,int> cnt;

for(char c:s)
{
    cnt[c]++;
}
```

map 和 unordered_map 选择

考试:

需要排序:

↓

```
map
```

只需要快速查找:

↓

```
unordered_map
```

统计次数:

↓

```
unordered_map
```

第六章 **pair / tuple / iterator**（迭代器）

这一章属于 STL 基础连接部分。

很多 STL 函数：

```
sort()
find()
lower_bound()
map 遍历
```

都会用到。

一、**pair**（存两个数据）

头文件：

```
#include <utility>
```

通常：

```
#include <bits/stdc++.h>
```

已经包含。

1. **pair** 定义

名称

```
pair
```

用法

格式：

```
pair<类型1, 类型2> 名称;
```

例如：

```
pair<int, string> p;
```

表示：

存：

```
整数 + 字符串
```

初始化：

```
pair<int, string> p={1, "Tom"};
```

或者：

```
pair<int, string> p;
p.first=1;
p.second="Tom";
```

作用

把两个变量绑定在一起。

例如：

学生：

```
学号 + 姓名
```

坐标：

```
x + y
```

2. **first**

名称

```
first
```

用法

```
p.first;
```

例如：

```
pair<int, string> p={100, "Tom"};
cout<<p.first;
```

输出：

```
100
```

作用

访问 pair 第一个元素。

3. **second**

名称

```
second
```

用法

```
p.second;
```

例如：

```
cout<<p.second;
```

输出：

```
Tom
```

作用

访问第二个元素。

4. **make_pair()**

名称

```
make_pair()
```

用法

格式：

```
make_pair(数据1, 数据2);
```

例如：

```
auto p=make_pair(10, "hello");
```

等价：

```
pair<int, string> p={10, "hello"};
```

作用

快速创建 pair。

5. **pair** 比较

pair 可以直接比较。

例如：

```
pair<int,int> a={1,2};
pair<int,int> b={1,3};

cout<<(a<b);
```

比较规则：
先比较：

first

如果相同：
比较：

second

例如：

```
(1,2)
(1,3)
```

因为：

2 < 3

所以：

a < b

作用

常用于排序。

6. pair 排序

例如：

```
vector<pair<int,int>> v;

v.push_back({2,5});
v.push_back({1,3});
v.push_back({2,1});

sort(v.begin(),v.end());
```

结果：

```
(1,3)
(2,1)
(2,5)
```

规则：
第一关键字升序。
第二关键字升序。

7. 自定义 pair 排序

例如：
按照 second 排序：

```
bool cmp(pair<int,int>a,pair<int,int>b)
{
    return a.second<b.second;
}

sort(v.begin(),v.end(),cmp);
```

作用：
改变排序规则。

二、tuple（三个及以上数据）

头文件：

```
#include <tuple>
```

1. tuple 定义

名称

tuple

用法

```
tuple<int,string,double> t;
```

表示：

```
int
string
double
```

三个数据。

初始化：

```
tuple<int,string,double> t
=
{
    1,
    "Tom",
    99.5
};
```

作用

存储多个不同类型的数据。

pair 只能两个。
tuple 可以多个。

2. get()

名称

get<>

用法

格式：

```
get<编号>(tuple);
```

例如：

```
tuple<int,string,double> t={1,"Tom",99.5};
```

```
cout<<get<0>(t);
```

输出：

1

```
cout<<get<1>(t);
```

输出：

Tom

注意：
编号从 0 开始。

```
get<0>
第一个
get<1>
```

第二个
`get<2>`
第三个

作用

访问 `tuple` 元素。

三、**iterator**（迭代器）

`iterator` 是 STL 的核心。
简单理解：
迭代器 = 指向容器元素的指针。

类似：
C 语言：

```
int *p;
```

STL：

```
vector<int>::iterator it;
```

1. **iterator** 定义

名称

```
iterator
```

用法

`vector`：

```
vector<int>::iterator it;
```

例如：

```
vector<int> v={1,2,3};  
vector<int>::iterator it;
```

现在推荐：

```
auto it;
```

例如：

```
auto it=v.begin();
```

作用

遍历容器。

2. **begin()**

名称

```
begin()
```

用法

```
auto it=v.begin();
```

例如：

```
vector<int> v={1,2,3};
```

```
auto it=v.begin();  
cout<<*it;
```

输出：

```
1
```

作用

返回第一个元素位置。

3. **end()**

名称

```
end()
```

用法

```
auto it=v.end();
```

注意：
`end` 不是最后元素。

它指向：

```
最后一个元素后面
```

所以：

错误：

```
cout<<*v.end();
```

正确：

```
cout<<*(v.end()-1);
```

作用

表示遍历结束。

4. 解引用 `*`

名称

```
*
```

用法

```
*iterator
```

例如：

```
auto it=v.begin();  
cout<<*it;
```

输出：

```
第一个元素
```

作用

获得迭代器指向的数据。

5. iterator 移动

名称

++

-

用法

向后:

```
it++;
```

向前:

```
it--;
```

例如:

```
auto it=v.begin();
it++;
cout<<*it;
```

输出:

第二个元素。

6. iterator 遍历 vector

方法

```
for(auto it=v.begin();
     it!=v.end();
     it++)
{
    cout<<*it;
}
```

输出:

```
123
```

7. iterator 遍历 set

set 不能:

```
s[0]
```

只能:

```
for(auto it=s.begin();
     it!=s.end();
     it++)
{
    cout<<*it;
}
```

8. iterator 遍历 map (重点)

map 里面:

元素类型:

```
pair
```

例如:

```
map<string,int> mp;
```

```
mp["Tom"]=90;
mp["Bob"]=80;
```

```
for(auto it=mp.begin();
     it!=mp.end();
     it++)
```

```
{
    cout<<it->first;
    cout<<it->second;
}
```

注意:

这里:

```
it->first
```

等价:

```
(*it).first
```

9. 箭头符号 ->

名称

成员访问

用法

如果:

```
it
```

是指针:

访问成员:

```
it->成员
```

例如:

map:

```
it->first
it->second
```

作用

访问指针对象成员。

10. const_iterator

名称

```
const_iterator
```

用法

```
vector<int>::const_iterator it;
```

或者:

```
auto it=v.cbegin();
```

作用

只读迭代器。

不能:

```
*it=10;
```

11. reverse_iterator

名称

反向迭代器

用法

```
auto it=v.rbegin();
```

遍历:

```
for(auto it=v.rbegin();
    it!=v.rend();
    it++)
{
    cout<<*it;
}
```

例如:

vector:

```
1 2 3
```

输出:

```
321
```

作用

反向遍历。

四、auto 关键字（STL 必用）

名称

```
auto
```

用法

不用写复杂类型:

传统:

```
vector<int>::iterator it;
```

auto:

```
auto it=v.begin();
```

map:

传统:

```
map<string,int>::iterator it;
```

auto:

```
auto it=mp.begin();
```

作用

让编译器自动推断类型。
考试非常推荐。

五、范围 for 循环（C++11）

名称

range-based for

用法

vector:

```
for(auto x:v)
{
    cout<<x;
}
```

string:

```
for(char c:s)
{
    cout<<c;
}
```

map:

```
for(auto p:mp)
{
    cout<<p.first;
    cout<<p.second;
}
```

作用

快速遍历。

六、STL 常用头文件总表（考试）

功能	头文件
vector	string
stack	<stack>
queue	<queue>
priority_queue	<queue>
set/map	<set> <map>
算法	<algorithm>
累加	<numeric>
pair	<utility>
tuple	<tuple>

第七章 C++ STL 上机考试万能模板

这一部分不是介绍函数，而是整理考试直接能套用的代码模板。

1. 万能头文件模板

名称

```
#include <bits/stdc++.h>
```

用法

```
#include <bits/stdc++.h>
using namespace std;

int main()
{

    return 0;
}
```

作用

一次包含几乎所有 STL:

包括:

```
vector
string
map
set
queue
stack
algorithm
numeric
```

等。

考试最推荐。

2. 快速输入输出

名称

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

用法

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

}
```

作用

关闭 C 和 C++ 输入同步，提高速度。
普通题可以不写。
数据量大建议写。

3. vector 输入模板

场景

输入 n 个数字。

写法

```
int n;
cin>>n;

vector<int> v(n);

for(int i=0;i<n;i++)
{
    cin>>v[i];
}
```

作用

快速创建数组。

另一种：

```
vector<int> v;

for(int i=0;i<n;i++)
{
    int x;
    cin>>x;
    v.push_back(x);
}
```

区别：

方法	特点
vector(n)	提前知道数量
push_back	不知道数量

4. vector 遍历模板

方法 1：下标

```
for(int i=0;i<v.size();i++)
{
    cout<<v[i]<<" ";
}
```

适合：
需要下标。

方法 2：范围 for

```
for(auto x:v)
{
    cout<<x<<" ";
}
```

适合：
只读取。

方法 3：引用

```
for(auto &x:v)
{
    cin>>x;
}
```

作用：
直接修改原元素。

例如：

```
for(auto &x:v)
{
    x*=2;
}
```

5. 数组排序模板

升序

```
sort(a,a+n);
```

例如：

```
int a[5]={3,1,5,2,4};
sort(a,a+5);
```

结果：

```
1 2 3 4 5
```

vector 排序

```
sort(v.begin(),v.end());
```

降序

数组：

```
sort(a,a+n,greater<int>());
```

vector：

```
sort(v.begin(),v.end(),greater<int>());
```

6. 字符串排序模板

名称

字符排序

用法

```
string s="dcba";

sort(s.begin(),s.end());
```

结果:

abcd

降序:

```
sort(s.begin(),s.end(),greater<char>());
```

结果:

dcba

7. vector 求最大最小值

最大值

```
int mx=*max_element(v.begin(),v.end());
```

最小值

```
int mn=*min_element(v.begin(),v.end());
```

最大值位置

```
int pos=max_element(v.begin(),v.end())-v.begin();
```

例如:

v={3,7,2}

返回:

1

8. vector 求和模板

名称

accumulate

用法

```
int sum=accumulate(
    v.begin(),
    v.end(),
    0
);
```

例如:

v={1,2,3}

结果:

6

9. 去重模板

场景

数组:

1 2 2 3 3

变:

1 2 3

写法

```
sort(v.begin(),v.end());

v.erase(
    unique(v.begin(),v.end()),
    v.end()
);
```

步骤:

第一步:

排序:

1 2 2 3 3

第二步:

unique:

1 2 3 ? ?

第三步:

erase 删除。

10. 查找元素模板

方法 1: find

```
auto it=find(
    v.begin(),
    v.end(),
    x
);

if(it!=v.end())
{
    cout<<"找到";
}
```

方法 2: count

```
if(count(v.begin(),v.end(),x))
{
    cout<<"存在";
}
```

11. set 去重模板

场景

自动排序去重。

代码:

```
set<int> s;

for(int x:v)
{
    s.insert(x);
}
```

遍历:

```
for(auto x:s)
{
    cout<<x<<" ";
}
```

12. map 统计次数模板（超级高频）

场景

统计：

- 字符出现次数
- 数字出现次数
- 单词次数

代码：

```
map<int,int> mp;
```

```
for(int x:v)
{
    mp[x]++;
}
```

例如：

输入：

```
1 2 2 3
```

结果：

```
1 -> 1次
2 -> 2次
3 -> 1次
```

字符串：

```
string s="hello";
```

```
map<char,int> mp;
```

```
for(char c:s)
{
    mp[c]++;
}
```

结果：

```
h 1
e 1
l 2
o 1
```

13. unordered_map 统计模板

推荐：

```
unordered_map<int,int> mp;
```

```
for(int x:v)
{
    mp[x]++;
}
```

速度更快。

14. map 遍历模板

方法 1

```
for(auto p:mp)
{
    cout<<p.first<<" ";
}
```

```
    cout<<p.second;
}
```

方法 2（修改）

```
for(auto &p:mp)
{
    p.second++;
}
```

15. stack 括号匹配模板

场景

判断：

```
(( ))
```

是否合法。

代码：

```
string s;

cin>>s;

stack<char> st;

for(char c:s)
{
    if(c=='(')
    {
        st.push(c);
    }

    else
    {
        if(st.empty())
        {
            cout<<"NO";

            return 0;
        }

        st.pop();
    }
}

if(st.empty())
    cout<<"YES";

else
    cout<<"NO";
```

16. queue BFS 模板

场景

图搜索：

- 迷宫
- 最短路径
- 层序遍历

模板：

```
queue<int> q;
```

```
q.push(start);
```

```
while(!q.empty())
{
    int x=q.front();

    q.pop();

    //处理x
}
```

17. priority_queue 最大值模板

默认大根堆

```
priority_queue<int> pq;

pq.push(5);
pq.push(10);
pq.push(3);

cout<<pq.top();
```

输出:

10

18. priority_queue 小根堆

```
priority_queue<
    int,
    vector<int>,
    greater<int>
> pq;
```

例如:

```
pq.push(5);
pq.push(2);
pq.push(8);
```

top:

2

19. 二分查找模板

前提:
必须排序。

lower_bound

```
auto it=
lower_bound(
    v.begin(),
    v.end(),
    x
);

int pos=it-v.begin();
```

寻找:

>=x

upper_bound

```
auto it=
upper_bound(
    v.begin(),
    v.end(),
    x
);
```

寻找:

>x

20. 全排列模板

next_permutation

例如:

```
string s="abc";

do
{
    cout<<s<<endl;
}
while(next_permutation(
    s.begin(),
    s.end()
));
```

输出:

abc
acb
bac
bca
cab
cba

21. 二维 vector 模板

定义

```
vector<vector<int>>> v(
    n,
    vector<int>(m)
);
```

创建:

n 行 m 列。

例如:

```
vector<vector<int>>> a(
    3,
    vector<int>(4,0)
);
```

得到:

3×4 全 0 矩阵。

22. 二维 vector 遍历

```
for(int i=0;i<v.size();i++)
{
    for(int j=0;j<v[i].size();j++)
    {
        cout<<v[i][j];
    }
}
```

23. pair 排序模板

默认

```
vector<pair<int,int>> v;  
  
sort(v.begin(),v.end());
```

排序：
第一关键字
↓
第二关键字。

按第二关键字

```
sort(  
v.begin(),  
v.end(),  
[](pair<int,int>a,pair<int,int>b)  
{  
    return a.second<b.second;  
});
```

24. 常用字符串处理组合

删除某字符

例如删除空格：

```
s.erase(  
remove(  
s.begin(),  
s.end(),  
' '  
)  
s.end()  
);
```

字符串反转

```
reverse(  
s.begin(),  
s.end()  
);
```

字符串转数字

```
int x=stoi(s);
```

数字转字符串

```
string s=to_string(x);
```

25. STL 复杂度速记

操作	复杂度
vector 尾插 push_back	O(1)
vector 中间插入	O(n)
sort	O(nlogn)
find(vector)	O(n)
binary_search	O(logn)
set 查找	O(logn)
map 查找	O(logn)
unordered_map 查找	平均 O(1)
priority_queue 插入	O(logn)

26. 考试最常用 TOP20 函数（建议背）

vector:
push_back()
pop_back()
size()
erase()
insert()
clear()
begin()
end()

string:
getline()
size()
substr()
find()
erase()
replace()
append()

algorithm:
sort()
reverse()
find()
count()
accumulate()
max_element()
min_element()
lower_bound()
upper_bound()

map:
insert
[]
find
count

stack:
push
pop
top

queue:
push
pop
front

priority_queue:
push
pop
top

第八章 STL 补充知识（容易遗漏但考试可能出现）

这一章主要补充：

- array
- unordered_set
- bitset
- numeric 更多函数
- functional
- lambda 表达式
- 自定义比较器
- STL 常见坑

一、array（固定大小数组）

头文件：

```
#include <array>
```

1. array 定义

名称

```
array
```

用法

格式:

```
array<类型,大小> 名称;
```

例如:

```
array<int,5> a;
```

表示:

```
长度固定为5的int数组
```

初始化:

```
array<int,5> a={1,2,3,4,5};
```

作用

STL 版本的普通数组。

区别:

普通数组:

```
int a[5];
```

array:

```
array<int,5> a;
```

优点:

- 支持 STL 函数
- 有 size()
- 有迭代器

2. size()

名称

```
size()
```

用法

```
a.size();
```

例如:

```
array<int,5> a;
```

```
cout<<a.size();
```

输出:

```
5
```

作用

获取数组大小。

3. front()

用法

```
a.front();
```

例如:

```
cout<<a.front();
```

作用:

访问第一个元素。

4. back()

用法

```
a.back();
```

作用:

访问最后元素。

5. fill()

用法

```
a.fill(0);
```

例如:

```
array<int,5> a;
```

```
a.fill(10);
```

结果:

```
10 10 10 10 10
```

作用

全部赋值。

6. begin()

用法

```
a.begin();
```

作用:

返回首迭代器。

7. end()

用法

```
a.end();
```

作用:

返回尾后迭代器。

二、unordered_set（哈希集合）

头文件:

```
#include <unordered_set>
```

1. unordered_set 定义

名称

```
unordered_set
```

用法

```
unordered_set<int> s;
```

作用

类似 `set`。

区别：

	set	unordered_set
底层	红黑树	哈希表
排序	自动排序	无序
速度	O(logn)	平均 O(1)

2. insert()

用法

```
s.insert(x);
```

例如：

```
unordered_set<int> s;  
s.insert(3);  
s.insert(1);
```

输出顺序不固定。

3. find()

用法

```
auto it=s.find(x);
```

判断：

```
if(it!=s.end())  
{  
    cout<<"存在";  
}
```

4. count()

用法

```
s.count(x);
```

返回：

存在：

```
1
```

不存在：

```
0
```

5. erase()

用法

```
s.erase(x);
```

作用：

删除元素。

6. size()

用法

```
s.size();
```

作用：

元素数量。

三、bitset（二进制位操作）

头文件：

```
#include <bitset>
```

1. bitset 定义

名称

```
bitset
```

用法

```
bitset<8> b;
```

表示：

8 位二进制。

初始化：

```
bitset<8> b("1010");
```

结果：

```
00001010
```

作用

处理二进制非常方便。

例如：

- 位运算
- 状态压缩

2. count()

用法

```
b.count();
```

例如：

```
bitset<8> b("1011");  
cout<<b.count();
```

输出：

```
3
```

作用

统计 1 的数量。

3. size()

用法

```
b.size();
```

作用：
位数。

4. test()

名称

```
test()
```

用法

```
b.test(pos);
```

例如：

```
bitset<8> b("1010");  
cout<<b.test(1);
```

作用：
判断第 pos 位是否为 1。

5. set()

用法

全部置 1：

```
b.set();
```

指定位置：

```
b.set(2);
```

作用：
把某位变成 1。

6. reset()

用法

全部置 0：

```
b.reset();
```

指定位置：

```
b.reset(2);
```

作用：
把某位变成 0。

7. flip()

用法

全部取反：

```
b.flip();
```

指定：

```
b.flip(2);
```

例如：

0→1
1→0

8. to_string()

用法

```
string s=b.to_string();
```

作用：
转换成字符串。

9. to_ulong()

用法

```
unsigned long x=b.to_ulong();
```

作用：
转换成整数。

四、numeric 头文件其他函数

头文件：

```
#include <numeric>
```

1. accumulate()

前面讲过：

```
accumulate(  
begin,  
end,  
初始值  
);
```

作用：
求和。

2. iota()

名称

```
iota()
```

用法

```
iota(  
begin,  
end,  
初始值  
);
```

例如：

```
vector<int> v(5);  
  
iota(  
v.begin(),  
v.end(),  
1  
);
```

结果：

```
1 2 3 4 5
```

作用

连续赋值。

类似:

```
for 循环赋值
```

3. inner_product()

名称

```
inner_product()
```

用法

```
inner_product(  
a.begin(),  
a.end(),  
b.begin(),  
0  
);
```

例如:

```
a:  
1 2 3
```

b:

```
4 5 6
```

计算:

```
1*4+2*5+3*6
```

结果:

```
32
```

作用

计算向量点积。

4. partial_sum()

名称

```
partial_sum()
```

用法

```
partial_sum(  
begin,  
end,  
输出位置  
);
```

例如:

```
vector<int> v={1,2,3,4};  
vector<int> ans(4);
```

```
partial_sum(  
v.begin(),  
v.end(),  
ans.begin()  
);
```

结果:

```
1 3 6 10
```

作用

前缀和。

5. adjacent_difference()

名称

```
adjacent_difference()
```

用法

```
adjacent_difference(  
begin,  
end,  
输出位置  
);
```

例如:

输入:

```
1 3 6 10
```

输出:

```
1 2 3 4
```

作用

求相邻差。

五、functional（函数对象）

头文件:

```
#include <functional>
```

常用于:

sort 第三个参数。

1. greater()

名称

```
greater<>
```

用法

降序:

```
sort(  
v.begin(),  
v.end(),  
greater<int>()  
);
```

作用:

从大到小排序。

2. less()

默认:

```
less<int>()
```

用法:

```
sort(  
v.begin(),  
v.end(),  
less<int>()  
);
```

作用:

升序排序。

3. plus()

用法

```
plus<int>()
```

作用：
加法。

4. minus()

```
minus<int>()
```

作用：
减法。

5. multiplies()

```
multiplies<int>()
```

作用：
乘法。

6. divides()

```
divides<int>()
```

作用：
除法。

六、lambda 表达式（非常重要）

C++11 以后：
可以直接写匿名函数。

1. 基本格式

```
[捕获](参数)
{
    函数体
}
```

2. sort 中使用（最常见）

例如：
按照升序：

```
sort(
v.begin(),
v.end(),
[](int a,int b)
{
    return a<b;
}
);
```

等价：

```
sort(v.begin(),v.end());
```

3. 降序

```
sort(
v.begin(),
v.end(),
[](int a,int b)
{
    return a>b;
}
);
```

4. pair 排序

按照 second:

```
sort(
v.begin(),
v.end(),
[](pair<int,int>a,pair<int,int>b)
{
    return a.second<b.second;
}
);
```

5. 自定义规则

例如：
奇数在前，偶数在后：

```
sort(
v.begin(),
v.end(),
[](int a,int b)
{
    return a%2>b%2;
}
);
```

七、STL 常见坑

1. vector 删除元素

错误：

```
for(auto x:v)
{
    if(x==3)
        v.erase(...);
}
```

原因：
删除导致迭代器失效。

正确：

```
for(auto it=v.begin();
it!=v.end();)
{
    if(*it==3)
        it=v.erase(it);

    else
        it++;
}
```

2. vector 访问越界

错误：

```
vector<int> v;

v[0]=10;
```

因为：
没有空间。

正确：

```
v.push_back(10);
```

或者：

```
vector<int> v(10);
```

3. map 中 [] 自动创建

例如：

```
map<string,int> mp;
```

```
cout<<mp["Tom"];
```

即使不存在：
也会创建：

```
Tom -> 0
```

如果只是查询：
用：

```
find()
```

4. sort 区间

错误：

```
sort(v.begin(),v.end()-1);
```

少排序最后一个。

正确：

```
sort(v.begin(),v.end());
```

5. unique 必须配 erase

错误：

```
unique(v.begin(),v.end());
```

不会删除。

正确：

```
v.erase(
unique(v.begin(),v.end()),
v.end()
);
```

6. lower_bound 必须有序

错误：

```
vector<int> v={5,1,3};
```

```
lower_bound(v.begin(),v.end(),3);
```

结果错误。

必须：

```
sort(v.begin(),v.end());
```

7. priority_queue 不能遍历

错误：

```
for(auto x:pq)
```

priority_queue 没有迭代器。

只能：

```
while(!pq.empty())
{
    cout<<pq.top();
    pq.pop();
}
```

第九章 C++ STL 函数速查表（机考打印版）

这一章按照考试查阅习惯整理。

格式：

名称 → 完整写法 → 参数怎么填 → 作用 → 示例

一、vector 速查表（重点）

头文件：

```
#include <vector>
```

定义：

```
vector<int> v;
```

1. push_back()

完整写法

```
v.push_back(x);
```

参数

一个元素：

```
x
```

例如：

```
v.push_back(10);
```

作用

尾部添加元素。

2. pop_back()

完整写法

```
v.pop_back();
```

参数

无。

作用

删除最后一个元素。

3. size()

完整写法

```
v.size();
```

参数

无。

返回

元素个数。

例如：

```
cout<<v.size();
```

4. empty()

完整写法

```
v.empty();
```

参数

无。

返回

```
true / false
```

作用：
判断是否为空。

5. clear()

完整写法

```
v.clear();
```

参数：

无。

作用：
清空 vector。

6. front()

完整写法

```
v.front();
```

参数：

无。

作用：
访问第一个元素。

7. back()

完整写法

```
v.back();
```

参数：

无。

作用：
访问最后元素。

8. begin()

完整写法

```
v.begin();
```

参数：

无。

作用：
返回首迭代器。

9. end()

完整写法

```
v.end();
```

参数：

无。

作用：
返回尾后迭代器。

10. insert()

完整写法

```
v.insert(pos,value);
```

参数：

两个：

□ 插入位置

□ 插入值

例如：

```
v.insert(v.begin()+2,100);
```

作用：

指定位置插入。

多个元素：

```
v.insert(pos,n,value);
```

例如：

```
v.insert(v.begin(),3,5);
```

结果：

```
5 5 5 ...
```

作用：

插入 n 个相同元素。

11. erase()

完整写法 1

删除一个：

```
v.erase(pos);
```

参数：

迭代器。

例如：

```
v.erase(v.begin()+2);
```

作用：

删除指定位置。

完整写法 2

删除区间：

```
v.erase(first,last);
```

例如：

```
v.erase(v.begin()+1,v.begin()+3);
```

删除：

下标 1 到 2。

12. resize()

完整写法

```
v.resize(n);
```

参数：

新大小。

例如：

```
v.resize(10);
```

作用：

改变 vector 长度。

填充值：

```
v.resize(10,0);
```

不足：

补 0。

13. capacity()

完整写法

```
v.capacity();
```

参数：

无。

作用：

返回当前容量。

注意：

不是元素数量。

区别：

```
size()
```

实际元素数量。

```
capacity()
```

申请空间大小。

14. reserve()

完整写法

```
v.reserve(n);
```

参数：

预留容量。

例如：

```
v.reserve(100);
```

作用：

提前申请空间。

减少扩容次数。

15. shrink_to_fit()

完整写法

```
v.shrink_to_fit();
```

参数：

无。

作用：

尝试释放多余容量。

16. at()

完整写法

```
v.at(index);
```

参数：

下标。

例如：

```
v.at(2);
```

作用：

访问元素。

区别：

```
v[2]
```

越界不检查。

```
v.at(2)
```

越界抛异常。

二、string 速查表（重点）

头文件：

```
#include <string>
```

定义：

```
string s;
```

1. size()

```
s.size();
```

作用：

字符串长度。

2. length()

```
s.length();
```

作用：

字符串长度。

和 size 完全一样。

3. empty()

```
s.empty();
```

作用：
判断字符串为空。

4. clear()

```
s.clear();
```

作用：
清空字符串。

5. push_back()

```
s.push_back(c);
```

参数：
字符。
例如：

```
s.push_back('a');
```

作用：
尾部添加字符。

6. pop_back()

```
s.pop_back();
```

作用：
删除最后字符。

7. append()

写法

```
s.append(str);
```

例如：

```
s="hello";  
s.append(" world");
```

结果：

```
hello world
```

作用：
追加字符串。

8. substr()

写法

```
s.substr(pos, len);
```

两个参数：
☐ 起始位置
☐ 长度

例如：

```
string s="abcdef";
```

```
cout<<s.substr(1,3);
```

结果：

```
bcd
```

作用：
截取子串。

9. find()

写法

```
s.find(str);
```

参数：
要查找内容。

例如：

```
s.find("abc");
```

返回：
第一次出现位置。

不存在：

```
string::npos
```

10. replace()

写法

```
s.replace(pos, len, str);
```

参数：
三个：
☐ 开始位置
☐ 替换长度
☐ 新字符串

例如：

```
s.replace(1,2,"xx");
```

作用：
替换字符串。

11. erase()

删除位置

```
s.erase(pos, len);
```

例如：

```
s.erase(1,3);
```

作用：
删除指定字符。

12. insert()

写法

```
s.insert(pos, str);
```

例如：

```
s.insert(2,"abc");
```

作用：
插入字符串。

13. compare()

写法

```
s.compare(str);
```

作用：
比较字符串。

返回：

```
0
```

相等。

14. getline()

写法

```
getline(cin,s);
```

作用：
读取一整行。

区别：

```
cin>>
```

遇空格停止。

getline：
包含空格。

三、algorithm 速查表

头文件：

```
#include <algorithm>
```

1. sort()

写法

```
sort(begin,end);
```

参数：
两个迭代器。

例如：

```
sort(v.begin(),v.end());
```

作用：
排序。

降序：

```
sort(
v.begin(),
v.end(),
greater<int>()
);
```

2. reverse()

写法

```
reverse(begin,end);
```

作用：
反转。

3. max()

写法

```
max(a,b);
```

作用：
最大值。

4. min()

```
min(a,b);
```

作用：
最小值。

5. max_element()

写法

```
max_element(begin,end);
```

返回：
迭代器。

使用：

```
*max_element(v.begin(),v.end());
```

作用：
最大元素。

6. min_element()

```
*min_element(
v.begin(),
v.end()
);
```

作用：
最小元素。

7. find()

写法

```
find(begin,end,x);
```

参数：
三个：
□ 开始
□ 结束
□ 查找值

作用：
寻找元素。

8. count()

```
count(begin, end, x);
```

作用：
统计数量。

9. fill()

```
fill(begin, end, x);
```

作用：
全部填充。

10. replace()

```
replace(  
begin,  
end,  
old,  
new  
);
```

作用：
替换元素。

11. unique()

```
unique(begin, end);
```

作用：
删除连续重复。

常用：

```
v.erase(  
unique(v.begin(), v.end()),  
v.end()  
);
```

12. lower_bound()

```
lower_bound(  
begin,  
end,  
x  
);
```

要求：
有序。

作用：
第一个：

```
>=x
```

13. upper_bound()

```
upper_bound(  
begin,  
end,  
x  
);
```

作用：
第一个：

```
>x
```

14. binary_search()

```
binary_search(  
begin,  
end,  
x  
);
```

作用：
二分查找是否存在。

返回：
true/false。

15. next_permutation()

```
next_permutation(  
begin,  
end  
);
```

作用：
下一个排列。

四、numeric 速查表

头文件：

```
#include <numeric>
```

1. accumulate()

写法

```
accumulate(  
begin,  
end,  
init  
);
```

参数：

三个：

- ☐ 开始
- ☐ 结束
- ☐ 初始值

例如：

```
accumulate(  
v.begin(),  
v.end(),  
0  
);
```

作用：

求和。

2. iota()

```
iota(  
begin,  
end,  
start  
);
```

作用：
连续赋值。

例如：

```
iota(v.begin(), v.end(), 1);
```

结果：

```
1 2 3...
```

3. partial_sum()

```
partial_sum(  
begin,  
end,  
out  
);
```

作用：
前缀和。

五、map 速查表

定义：

```
map<int, int> mp;
```

1. []

```
mp[key];
```

作用：
访问 value。
不存在自动创建。

2. insert()

```
mp.insert({key, value});
```

作用：
插入。

3. erase()

```
mp.erase(key);
```

作用：
删除。

4. find()

```
mp.find(key);
```

作用：
查找 key。

5. count()

```
mp.count(key);
```

作用：
判断 key 是否存在。

6. 遍历

```
for(auto p:mp)  
{  
    cout<<p.first;  
    cout<<p.second;
```

```
}
```

六、stack 速查表

定义：

```
stack<int> st;
```

push:

```
st.push(x);
```

入栈。

pop:

```
st.pop();
```

删除栈顶。

top:

```
st.top();
```

访问栈顶。

empty:

```
st.empty();
```

判断空。

七、queue 速查表

定义：

```
queue<int> q;
```

push:

```
q.push(x);
```

入队。

pop:

```
q.pop();
```

出队。

front:

```
q.front();
```

队头。

back:

```
q.back();
```

队尾。

八、priority_queue 速查表

定义：

```
priority_queue<int> pq;
```

```
push:
pq.push(x);
```

```
top:
pq.top();
```

作用：
最大值。

```
pop:
pq.pop();
```

```
小根堆:
priority_queue<
int,
vector<int>,
greater<int>
> pq;
```

九、set 速查表

```
定义:
set<int> s;
```

```
插入:
s.insert(x);
```

```
删除:
s.erase(x);
```

```
查找:
s.find(x);
```

```
数量:
s.size();
```

```
判断存在:
s.count(x);
```

十、考试最常忘参数总结

第十章 C++ 常用头文件速查（cmath + 其他高频库）

这一部分整理上机考试除了 STL 之外最常用的函数库。

重点：

- <cmath> 数学函数（重点）
- <iomanip> 格式化输出
- <cstdlib> 常用工具函数
- <cstring> C 字符串处理
- <ctime> 时间函数
- <numeric> 数学运算
- <sstream> 字符串流
- <fstream> 文件操作（了解）
- <random> 随机数

函数	参数
sort	开始，结束，比较函数
reverse	开始，结束
find	开始，结束，值
count	开始，结束，值
fill	开始，结束，值
accumulate	开始，结束，初始值
lower_bound	开始，结束，值
upper_bound	开始，结束，值
substr	位置，长度
erase(vector)	位置或区间
insert(vector)	位置，值
replace(string)	位置，长度，新字符串
resize	新大小
reserve	容量

- <limits> 极值

一、cmath（数学函数库）【必背】

头文件：

```
#include <cmath>
```

作用：
提供数学计算函数。

1. abs()

名称

```
abs()
```

用法

```
abs(x);
```

参数：
一个数字。

例如：

```
int x=-10;
cout<<abs(x);
```

输出：

```
10
```

作用

求绝对值。

注意：

整数：

```
abs()
```

浮点：

推荐：

```
fabs()
```

2. fabs()

名称

```
fabs()
```

用法

```
fabs(x);
```

例如:

```
double x=-3.14;
cout<<fabs(x);
```

输出:

```
3.14
```

作用

求浮点数绝对值。

3. sqrt()

名称

```
sqrt()
```

用法

```
sqrt(x);
```

例如:

```
cout<<sqrt(9);
```

输出:

```
3
```

浮点:

```
cout<<sqrt(2.0);
```

输出:

```
1.414...
```

作用

平方根。

4. pow()

名称

```
pow()
```

用法

两个参数:

```
pow(a,b);
```

表示:

a的b次方

例如:

```
cout<<pow(2,3);
```

输出:

```
8
```

作用

幂运算。

常见:

平方:

```
pow(x,2)
```

立方:

```
pow(x,3)
```

5. exp()

名称

```
exp()
```

用法

```
exp(x);
```

表示:

e^x

例如:

```
cout<<exp(1);
```

约:

```
2.71828
```

作用

计算自然指数。

6. log()

名称

```
log()
```

用法

```
log(x);
```

表示:

$ln(x)$

例如:

```
cout<<log(2);
```

作用

自然对数。

7. log10()

名称

```
log10()
```

用法

```
log10(x);
```

例如:

```
cout<<log10(100);
```

输出:

```
2
```

作用

十进制对数。

8. sin()

名称

```
sin()
```

用法

```
sin(x);
```

注意:

参数单位:

弧度

不是角度。

例如:

```
sin(M_PI/2);
```

结果:

```
1
```

作用

正弦。

9. cos()

用法

```
cos(x);
```

作用:

余弦。

10. tan()

用法

```
tan(x);
```

作用:

正切。

11. asin()

名称

```
asin()
```

作用:

反正弦。

返回:

弧度。

12. acos()

```
acos(x);
```

作用:

反余弦。

13. atan()

```
atan(x);
```

作用:

反正切。

14. atan2()

【重点】

用法

```
atan2(y,x);
```

返回:

点:

(x,y)

的角度。

例如:

```
double angle=atan2(y,x);
```

作用

计算方向角。

常用于:

- 几何
- 向量

15. ceil()

名称

```
ceil()
```

用法

```
ceil(x);
```

例如:

```
ceil(3.2);
```

结果:

```
4
```

作用

向上取整。

16. floor()

名称

```
floor()
```

用法

```
floor(x);
```

例如:

```
floor(3.8);
```

结果:

```
3
```

作用

向下取整。

17. round()

名称

```
round()
```

用法

```
round(x);
```

例如:

```
round(3.5);
```

结果:

```
4
```

作用

四舍五入。

18. trunc()

名称

```
trunc()
```

用法

```
trunc(x);
```

例如:

```
trunc(3.8);
```

结果:

```
3
```

作用

截断小数。

19. fmod()

名称

```
fmod()
```

用法

```
fmod(a,b);
```

例如:

```
fmod(5.5,2);
```

结果:

```
1.5
```

作用

浮点取余。

类似:

整数:

```
a%b
```

浮点:

```
fmod(a,b)
```

20. hypot()

名称

```
hypot()
```

用法

```
hypot(a,b);
```

计算:

$\sqrt{a^2 + b^2}$

例如：

```
hypot(3,4);
```

结果：

```
5
```

作用

求直角三角形斜边。

cmath 常量

M_PI

圆周率：

```
M_PI
```

例如：

```
double r=2;
double area=M_PI*r*r;
```

二、**iomanip**（格式化输出）【重要】

头文件：

```
#include <iomanip>
```

1. setprecision()

名称

```
setprecision()
```

用法

```
cout<<setprecision(n);
```

例如：

```
cout<<setprecision(3)<<3.14159;
```

输出：

```
3.14
```

默认：

控制有效数字。

配合：

```
fixed
```

2. fixed

用法

```
cout<<fixed;
```

作用：

固定小数位。

3. setprecision + fixed

【必背】

```
cout
<<fixed
<<setprecision(2)
<<3.14159;
```

输出：

```
3.14
```

考试最常用。

4. setw()

名称

```
setw()
```

用法

```
setw(n)
```

例如：

```
cout<<setw(5)<<123;
```

输出：

```
123
```

作用

设置输出宽度。

5. left

```
cout<<left;
```

作用：

左对齐。

6. right

```
cout<<right;
```

作用：

右对齐。

7. setfill()

用法

```
setfill('*')
```

例如：

```
cout
<<setfill('0')
<<setw(4)
<<12;
```

输出：

```
0012
```

三、cstring（C 风格字符串）

头文件：

```
#include <cstring>
```

处理：

```
char 数组
```

1. strlen()

名称

```
strlen()
```

用法

```
strlen(str);
```

例如：

```
char s[]="hello";  
  
cout<<strlen(s);
```

输出：

```
5
```

作用

计算字符串长度。

2. strcpy()

用法

```
strcpy(dest,src);
```

作用：

复制字符串。

3. strcat()

用法

```
strcat(a,b);
```

作用：

连接字符串。

4. strcmp()

用法

```
strcmp(a,b);
```

作用：

比较字符串。

返回：

```
0 相等  
<0 小于  
>0 大于
```

5. memset()

【必背】

用法

```
memset(数组,值,sizeof(数组));
```

例如：

```
int a[100];  
  
memset(a,0,sizeof(a));
```

作用：

快速初始化数组。

常用：

全部变：

0：

```
memset(a,0,sizeof(a));
```

-1：

```
memset(a,-1,sizeof(a));
```

注意：

不要：

```
memset(a,1,sizeof(a));
```

整数不会全部变 1。

四、cstdlib（常用工具）

头文件：

```
#include <cstdlib>
```

1. rand()

用法

```
rand();
```

作用：

生成随机数。

2. srand()

用法

```
srand(time(0));
```

作用：

设置随机种子。

通常：

```
srand(time(NULL));
```

3. atoi()

用法

```
atoi(str);
```

例如:

```
char s[]="123";

int x=atoi(s);
```

作用:
字符串转整数。

作用

字符串和数字互转。

4. exit()

用法

```
exit(0);
```

作用:
结束程序。

六、limits（极值）

头文件:

```
#include <limits>
```

1. INT_MAX

用法

```
INT_MAX
```

作用:
int 最大值。

约:

```
2147483647
```

2. INT_MIN

```
INT_MIN
```

作用:
int 最小值。

五、sstream（字符串流）

头文件:

```
#include <sstream>
```

1. stringstream

定义

```
stringstream ss;
```

字符串转数字

```
string s="123";

stringstream ss(s);

int x;

ss>>x;
```

结果:

```
x=123
```

3. numeric_limits

用法

```
numeric_limits<int>::max();
```

作用:
获取类型最大值。

例如:

```
long long INF=
numeric_limits<long long>::max();
```

数字转字符串

```
stringstream ss;

ss<<123;

string s;

ss>>s;
```

七、常用头文件总表（考试版）

头文件	作用	常用
cmath	数学计算	sqrt,pow,sin
iomanip	格式输出	fixed,setprecision
string	字符串	string
cstring	C 字符串	strlen,memset
sstream	字符串转换	stringstream
cstdlib	工具函数	rand,exit
ctime	时间	time
limits	极值	INT_MAX
numeric	数学算法	accumulate,iota
algorithm	算法	sort,find
vector	动态数组	vector
queue	队列	queue
stack	栈	stack
map	映射	map
set	集合	set

第十一章 C++ 常用语法速查（机考必备）

这一章不是 STL，而是写题时每天都会用到的 C++ 基础语法工具。

重点：

- 输入输出
- 类型转换
- 字符处理
- ASCII
- 位运算
- long long
- 常用技巧
- 防止错误

一、输入输出（iostream）

头文件：

```
#include <iostream>
```

1. cin 输入

用法

```
cin >> x;
```

例如：

```
int a;  
cin >> a;
```

作用：
读取变量。

2. 多个变量输入

```
int a,b,c;  
cin>>a>>b>>c;
```

输入：

```
1 2 3
```

结果：

```
a=1,b=2,c=3
```

3. cout 输出

用法

```
cout << x;
```

例如：

```
cout<<123;
```

输出：

```
123
```

4. 输出多个内容

```
cout<<"sum="<<sum;
```

5. 换行

方法 1

```
cout<<endl;
```

作用：
换行，并刷新缓冲区。

方法 2（推荐）

```
cout<<"\n";
```

速度更快。

考试推荐：

```
cout<<"\n";
```

6. getline()

【必背】

用法

```
string s;  
getline(cin,s);
```

输入：

```
hello world
```

读取：

```
hello world
```

注意

如果前面用了：

```
cin>>n;
```

后面：

```
getline(cin,s);
```

可能读到空行。

解决：

```
cin.ignore();
```

例如：

```
int n;  
cin>>n;  
cin.ignore();  
getline(cin,s);
```

二、类型转换（非常重要）

1. int 转 double

自动转换：

```
int a=5;  
double b=a;
```

结果:

5.0

2. 强制类型转换

写法 1

```
(double)a
```

例如:

```
int a=5,b=2;

cout<<(double)a/b;
```

结果:

2.5

写法 2 (C++ 推荐)

```
static_cast<double>(a)
```

例如:

```
double x=
static_cast<double>(a)/b;
```

3. string 转 int

stoi()

头文件:

```
#include <string>
```

用法:

```
string s="123";

int x=stoi(s);
```

结果:

123

4. string 转 long long

```
long long x=stoll(s);
```

5. string 转 double

```
double x=stod(s);
```

6. int 转 string

to_string()

用法:

```
string s=to_string(123);
```

结果:

"123"

三、字符处理 (ctype)

头文件:

```
#include <ctype>
```

1. isdigit()

名称

判断数字字符

用法

```
isdigit(c);
```

例如:

```
char c='5';

if(isdigit(c))
{
    cout<<"数字";
}
```

作用:

判断:

0-9

2. isalpha()

判断字母。

```
isalpha(c);
```

例如:

```
'a'
```

返回:

true。

3. islower()

判断小写。

```
islower(c);
```

4. isupper()

判断大写。

```
isupper(c);
```

5. tolower()

转换小写。

```
tolower(c);
```

例如:

```
char c='A';

c=tolower(c);
```

结果:

a

6. toupper()

转换大写。

```
toupper(c);
```

例如:

```
a→A
```

四、ASCII 码（高频）

字符本质：
数字。

1. 字符转数字

例如:

```
char c='5';
```

```
int x=c-'0';
```

结果:

```
5
```

原理:

ASCII:

```
'5'=53
```

```
'0'=48
```

```
53-48=5
```

2. 数字转字符

```
char c=x+'0';
```

例如:

```
int x=7;
```

```
char c=x+'0';
```

结果:

```
'7'
```

3. 字母大小写转换

大写:

```
char c='A';
```

```
c=c+32;
```

得到:

```
a
```

小写转大写:

```
c=c-32;
```

不过推荐:

```
toupper()  
tolower()
```

五、long long 使用（非常重要）

1. int 范围

```
-231 ~ 231-1
```

约:

```
21亿
```

2. long long 范围

约:

```
9×1018
```

3. 定义

```
long long x;
```

4. 常量

错误:

```
long long x=10000000000000;
```

可能溢出。

正确:

```
long long x=10000000000000LL;
```

六、三目运算符

名称

条件运算符

格式

```
条件 ? 真 : 假;
```

例如:

```
int max=  
a>b?a:b;
```

等价:

```
if(a>b)  
max=a;
```

```
else  
max=b;
```

七、位运算（了解）

符号:

```
&  
|  
^  
~
```

<<
>>

1. 按位与 &

例如:

5&3

二进制:

101
011

001

结果:

1

作用:

判断奇偶。

判断奇偶【常用】

偶数:

if((x&1)==0)

奇数:

if(x&1)

2. 按位或 |

a|b

作用:

合并二进制位。

3. 异或 ^

特点

相同:

0

不同:

1

应用:

交换变量:

a^=b;

b^=a;

a^=b;

4. 左移 «

x<<1

相当:

x*2

例如:

5<<1

结果:

10

5. 右移 »

x>>1

相当:

x/2

八、常用数学技巧

1. 最大公约数 gcd

头文件:

#include <numeric>

用法:

gcd(a,b);

例如:

cout<<gcd(12,8);

输出:

4

2. 最小公倍数 lcm

C++17:

lcm(a,b);

例如:

cout<<lcm(4,6);

输出:

12

3. 判断素数模板

```
bool isPrime(int n)
{
    if(n<2)
        return false;

    for(int i=2;i*i<=n;i++)
    {
        if(n%i==0)
            return false;
    }

    return true;
}
```

九、swap 交换

名称

swap()

头文件:

```
#include <algorithm>
```

用法

```
swap(a,b);
```

例如:

```
int a=1,b=2;
swap(a,b);
```

结果:

```
a=2,b=1
```

十、**auto** 自动类型推导

用法

```
auto x=10;
```

等价:

```
int x=10;
```

vector 遍历:

```
for(auto x:v)
{
}
```

map:

```
for(auto p:mp)
{
}
```

十一、引用 **&** (STL 常用)

普通变量

```
int a=10;
```

```
int &b=a;
```

b 就是 a 别名。

修改 **vector** 元素

错误:

```
for(auto x:v)
{
    x++;
}
```

不会改变。

正确:

```
for(auto &x:v)
{
    x++;
}
```

十二、**const** (防止修改)

定义:

```
const int x=10;
```

不能:

```
x=20;
```

常见:

```
const vector<int>& v
```

表示:

引用 **vector**, 但不能修改。

十三、宏定义 (了解)

1. 常量

```
#define PI 3.14159
```

2. 宏函数

```
#define MAX(a,b) ((a)>(b)?(a):(b))
```

现在 C++ 更推荐:

```
const
inline
```

十四、结构体 **struct**

考试经常用。

定义:

```
struct Student
{
    int id;

    string name;
};
```

使用:

```
Student s;
```

```
s.id=1;
```

```
s.name="Tom";
```

数组:

```
Student a[100];
```

vector:

```
vector<Student> v;
```

十五、自定义排序（结构体）

例如：

```
struct Node
{
    int x;

    int y;
};

bool cmp(Node a,Node b)
{
    return a.x<b.x;
}
```

使用：

```
sort(v.begin(),v.end(),cmp);
```

十六、万能 main 模板（考试推荐）

```
#include <bits/stdc++.h>

using namespace std;

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    return 0;
}
```

十七、考试最常用组合

1. 输入数组排序求和

```
vector<int> v(n);

for(auto &x:v)
    cin>>x;

sort(v.begin(),v.end());

int sum=
accumulate(v.begin(),v.end(),0);
```

2. 统计字符次数

```
map<char,int> mp;

for(char c:s)
{
    mp[c]++;
}
```

3. 删除重复元素

```
sort(v.begin(),v.end());

v.erase(
unique(v.begin(),v.end()),
v.end())
```

```
);
```

4. 保留两位小数

```
cout
<<fixed
<<setprecision(2)
<<x;
```

5. 最大最小值

```
int mx=*max_element(
v.begin(),
v.end()
);

int mn=*min_element(
v.begin(),
v.end()
);
```

最终考试头文件优先级

【必背】

```
#include <bits/stdc++.h>
```

里面全部包含。

重点掌握：

```
vector
string
algorithm
numeric
cmath
iomanip
map
set
queue
stack
sstream
cstring
```